



POLSKO-JAPOŃSKA AKADEMIA
TECHNIK KOMPUTEROWYCH

Programowanie aplikacji biznesowych

Katedra Inżynierii Oprogramowania

Wydział Informatyki

Jakub Głocki

s26523

**Wytworzenie systemu obsługi zamówień z użyciem
technologii chmurowych oraz mikroserwisów**

Praca Inżynierska

Promotor:
dr Krzysztof Barteczko

Warszawa, x 2026

Streszczenie

Niniejsza praca przedstawia projekt, implementację oraz wdrożenie w chmurze publicznej kompletnego systemu obsługi zamówień, zbudowanego w architekturze mikroserwisowej. System składa się z trzech mikroserwisów napisanych w języku Go — odpowiedzialnych za zamówienia, katalog produktów oraz płatności — komunikujących się synchronicznie poprzez interfejsy REST oraz asynchronicznie poprzez zdarzenia domenowe publikowane do brokera Apache Kafka, a także z aplikacji webowej (Vue 3) stanowiącej interfejs użytkownika. Warstwę danych zrealizowano na bazie PostgreSQL, a uwierzytelnianie — w oparciu o protokół OAuth 2.0 z rozszerzeniem PKCE i usługę Amazon Cognito.

System uruchomiono na zarządzanym klastrze Kubernetes (Amazon EKS), w ramach infrastruktury zdefiniowanej w całości jako kod (Terraform), objętej automatyzacją wdrożeń (GitHub Actions) oraz monitoringiem (Prometheus, Grafana). Działanie systemu zweryfikowano w docelowym środowisku chmurowym testami funkcjonalnymi i wydajnościowymi; w teście obciążeniowym system obsłużył kilkaset żądań na sekundę bez błędów, automatycznie skalując liczbę replik. Pracę uzupełniają analiza kosztów utrzymania środowiska oraz identyfikacja wąskich gardeł wraz z rekomendacjami optymalizacji.

Słowa kluczowe: mikroserwisy, Kubernetes, Amazon Web Services, architektura sterowana zdarzeniami, infrastruktura jako kod, Go

Spis treści

1	Wstęp	7
1.1	Cel i zakres pracy	8
1.2	Uzasadnienie wyboru tematu	9
1.3	Przegląd technologii chmurowych w kontekście mikroservisów	10
1.4	Struktura pracy	11
2	Teoretyczne podstawy architektury mikroservisowej	12
2.1	Architektura monolityczna a mikroservisowa	12
2.2	Charakterystyka mikroservisów	14
2.3	Komunikacja synchroniczna i asynchroniczna	15
2.3.1	Komunikacja synchroniczna — REST	15
2.3.2	Komunikacja asynchroniczna i architektura sterowana zdarzeniami	15
2.4	Wzorce projektowe w systemach mikroservisowych	16
2.5	Konteneryzacja i rola orkiestracji	17
3	Kubernetes jako platforma uruchomieniowa mikroservisów	19
3.1	Podstawowe obiekty Kubernetes	19
3.2	Zarządzanie konfiguracją i skalowaniem	20
3.3	Trwałe składowanie danych	21
3.4	Obserwowalność: Prometheus i Grafana	22
3.5	Mechanizmy bezpieczeństwa w Kubernetes	23

4	Amazon Web Services jako środowisko chmurowe	25
4.1	Charakterystyka AWS i jego architektury globalnej	25
4.2	Amazon EKS — zarządzana usługa Kubernetes	26
4.3	Usługi powiązane wykorzystane w projekcie	26
4.3.1	Amazon EC2 jako warstwa obliczeniowa	26
4.3.2	Amazon RDS for PostgreSQL jako warstwa danych	27
4.3.3	Amazon ECR — rejestr obrazów kontenerów	27
4.3.4	Amazon Cognito — uwierzytelnianie użytkowników	27
4.3.5	Amazon S3 i CloudFront — hosting aplikacji webowej	28
4.3.6	Amazon VPC, ALB i architektura sieciowa	28
4.3.7	IAM i Secrets Manager	28
5	Projekt systemu obsługi zamówień	30
5.1	Opis funkcjonalny systemu	30
5.2	Model domenowy	31
5.3	Architektura mikroservisowa systemu	33
5.4	Projekt API REST/JSON oraz kontraktów zdarzeń	34
5.4.1	Interfejsy REST	34
5.4.2	Kontrakty zdarzeń	35
5.5	Wybór technologii implementacyjnych — Go i Vue 3	36
5.6	Projekt bazy danych i persystencji — PostgreSQL	37
5.7	Asynchroniczna wymiana zdarzeń — Apache Kafka	39
5.8	Projekt uwierzytelniania i autoryzacji	39
6	Implementacja mikroservisów	41
6.1	Struktura projektu i wspólne wzorce serwisów	41
6.2	Serwis zamówień - logika biznesowa, maszyna stanów i idempotencja	42

6.3	Serwis produktów - katalog i rezerwacja stanów magazynowych	44
6.4	Serwis płatności - transakcyjna obsługa procesu płatności	44
6.5	Publikacja i konsumpcja zdarzeń Kafka	45
6.6	Walidacja tokenów JWT i integracja z Cognito	45
6.7	Obserwowalność serwisów	46
6.8	Konteneryzacja serwisów	47
6.9	Testy jednostkowe i integracyjne	48
7	Implementacja portalu obsługi zamówień	49
7.1	Architektura aplikacji frontendowej	49
7.2	Główne widoki i przepływy użytkownika	49
7.3	Warstwa komunikacji z API	53
7.4	Logowanie użytkownika — OAuth 2.0 z PKCE	54
8	Budowa środowiska chmurowego — infrastruktura jako kod	57
8.1	Terraform i zarządzanie stanem infrastruktury	57
8.2	Konfiguracja sieci VPC	57
8.3	Utworzenie klastra EKS i grup węzłów	59
8.4	Warstwa danych — RDS PostgreSQL i Secrets Manager	61
8.5	Wdrożenie mikroservisów i brokera Kafka do klastra	62
8.6	Routing ruchu — CloudFront, ALB i reverse proxy	63
8.7	Wdrożenie monitoringu — Prometheus i Grafana	65
8.8	Skalowanie aplikacji i zarządzanie cyklem życia	67
9	Testy i analiza działania systemu	69
9.1	Metodyka testowania	69
9.2	Testy funkcjonalne systemu	70
9.3	Testy wydajnościowe i skalowalność	71

9.3.1	Czasy odpowiedzi i efekt rozgrzania	71
9.3.2	Opóźnienie asynchronicznej walidacji	71
9.3.3	Automatyczne skalowanie pod obciążeniem	72
9.4	Analiza kosztowa wykorzystania usług AWS	73
9.4.1	Profilowanie procesora	74
9.4.2	Zidentyfikowane wąskie gardła	76
10	Wnioski i podsumowanie	78
10.1	Stopień realizacji celów pracy	78
10.2	Napotkane trudności i sposoby ich rozwiązania	79
10.3	Rekomendacje dotyczące dalszego rozwoju systemu	79
10.4	Podsumowanie końcowe	80
	Spis ilustracji	84
	Spis tabel	85
	Spis listingów	86
	Bibliografia	87

1. Wstęp

Handel elektroniczny w ostatniej dekadzie stał się jednym z najszybciej rozwijających się obszarów internetu. Systemy zamówień stanowią jego fundament przez co muszą sprostać z wieloma wyzwaniami. Takimi jak zmienne w czasie obciążenie, jak największa dostępność, krótkie czasy odpowiedzi oraz częstymi zmianami. Architektura monolityczna w której cała logika biznesowa zawiera się w jednej aplikacji okazuje się niewystarczająca. Zarówno pod względem skalowalności jak i tempa rozwoju oprogramowania.

Odpowiedzią przemysłu informatycznego na te ograniczenia stała się architektura mikro-serwisowa, polegająca na dekompozycji systemu na zbiór niewielkich, niezależnie wdrażanych usług, z których każda odpowiada za wyodrębniony fragment domeny biznesowej i komunikuje się z pozostałymi poprzez dobrze zdefiniowane interfejsy [1]. Podejście to, choć rozwiązuje wiele problemów monolitu, wprowadza jednocześnie nowe wyzwania: konieczność zarządzania komunikacją międzyusługową, zapewnienia spójności danych rozproszonych pomiędzy usługami, a także orkiestracji, monitorowania i skalowania wielu niezależnych komponentów [2]. W praktyce przemysłowej wyzwania te adresowane są przy użyciu konteneryzacji, platform orkiestracji, z spośród których standardem de facto stał się Kubernetes [3] oraz usług chmury publicznej, przejmujących od zespołów deweloperskich znaczną część odpowiedzialności za utrzymanie infrastruktury.

Niniejsza praca podejmuje się tego zagadnienia w ujęciu praktycznym. Jej przedmiotem jest wytworzenie kompletnego systemu obsługi zamówień zbudowanego w architekturze mikroserwisowej i wdrożonego w chmurze publicznej Amazon Web Services z wykorzystaniem zarządzanej usługi Kubernetes (Amazon EKS). Praca ta obejmuje pełny cykl wytwarzania oprogramowania. Zaczynając od analizy i projektu systemu. Idąc przez implementację aplikacji webowej i jej backendu. Z wytworzeniem infrastruktury chmurowej w podejściu *infrastructure as code*. Kończąc na testach i analizie działania systemu w środowisku docelowym (chmurowym).

1.1 Cel i zakres pracy

Celem pracy jest zaprojektowanie, implementacja oraz wdrożenie w chmurze publicznej Amazon Web Services systemu obsługi zamówień opartego na architekturze mikroserwisowej, a następnie weryfikacja jego działania w środowisku docelowym. Realizacja tak postawionego celu ma jednocześnie wykazać, że współczesne technologie chmurowe — zarządzana usługa Kubernetes, infrastruktura definiowana jako kod oraz zautomatyzowane procesy integracji i wdrażania — pozwalają pojedynczemu inżynierowi zbudować system o właściwościach (skalowalność, obserwowalność, bezpieczeństwo) tradycyjnie wymagających pracy całych zespołów utrzymaniowych.

Celem tej pracy jest zaprojektowanie, implementacja oraz wdrożenie w chmurze Amazon Web Services systemu obsługi zamówień opartego na architekturze mikroserwisowej. Weryfikacja poprawności działania systemu odbędzie się za pomocą testów manualnych oraz obciążeniowych. Rezultatem tej pracy będzie wykazanie jak za pomocą wymienioncy technologii można wytworzyć relatywnie szybko system z dobrą skalowalnością, obserwowalnością oraz bezpieczeństwem.

Cel główny można rozdzielić na:

- teoretyczna analiza podstaw architektury mikroserwisowej oraz związanymi z nią technologiami (konteneryzacja, Kubernetes, usługi chmurowe AWS),
- zaprojektowanie modelu domenowego, architektury usług oraz kontraktów komunikacyjnych systemu obsługi zamówień,
- implementacja mikroserwisów realizujących obsługę zamówień, katalog produktów oraz proces płatności,
- implementacja aplikacji webowej stanowiącej interfejs użytkownika systemu,
- budowa kompletnego środowiska uruchomieniowego w chmurze AWS w podejściu *infrastructure as code*,
- przeprowadzenie testów systemu oraz analiza jego działania i kosztów utrzymania w środowisku chmurowym.

Zakres pracy obejmuje wytworzenie następujących komponentów:

- trzech mikroserwisów zaimplementowanych w języku Go: serwisu zamówień (`order-service`), serwisu produktów (`product-service`) oraz serwisu płatności (`payment-service`), komunikujących się synchronicznie poprzez interfejsy

REST/JSON oraz asynchronicznie poprzez zdarzenia domenowe publikowane do brokera Apache Kafka,

- aplikacji webowej typu SPA (*single-page application*) zbudowanej w oparciu o framework Vue 3 i język TypeScript, umożliwiającej przeglądanie katalogu produktów, składanie zamówień oraz realizację płatności,
- warstwy persystencji opartej na relacyjnej bazie danych PostgreSQL, uruchamianej jako zarządzana usługa Amazon RDS,
- mechanizmów uwierzytelniania i autoryzacji użytkowników zrealizowanych z użyciem usługi Amazon Cognito i tokenów JWT,
- infrastruktury chmurowej zdefiniowanej w całości w narzędziu Terraform,
- podsystemu monitorowania opartego na narzędziach Prometheus i Grafana, uruchomionych wewnątrz klastra Kubernetes,

Poza zakresem pracy pozostają zagadnienia integracji z rzeczywistymi operatorami płatności (proces płatności jest symulowany), obsługa procesów logistycznych następujących po opłaceniu zamówienia oraz implementacja usług pomocniczych — takich jak powiadomienia czy analityka — które omówiono jedynie jako kierunki dalszego rozwoju systemu w rozdziale 10.

Poza zakresem pracy pozostaje implementacja usług pomocniczych jak powiadomienia oraz analityka. Integracja z rzeczywistymi operatorami płatności. Wytworzenie procesów ciągłej integracji za pomocą CI/CD zrealizowanych w GitHub Actions. Tematy te zostaną omówione jedynie jako kierunki dalszego rozwoju systemu w rozdziale 10.

1.2 Uzasadnienie wyboru tematu

Wybór tematu pracy wynika ze znaczenia omawianych technologii we współczesnym przemyśle informatycznym.

Architektura mikroserwisowa wraz z konteneryzacją i orkiestracją przestały być rozwiązaniami niszowymi, zarezerwowanymi dla największych firm technologicznych. Badania społeczności Cloud Native Computing Foundation wskazują, że zdecydowana większość organizacji wykorzystujących technologie chmurowe stosuje Kubernetes w środowiskach produkcyjnych, a odsetek ten systematycznie rośnie [4].

System zamówień jest bardzo dobrym tematem wykorzystania architektury mikroserwisowej. Dzieli się ona łatwo na odrębne tematy biznesowe jak katalog produktów, zamówienia oraz

płatności. Pomiędzy nimi występują interakcje synchroniczne jak i asynchroniczne. Wymusza to zmierzenie się z kluczowymi problemami systemów rozproszonych.

Literatura obszernie opisuje poszczególne technologie z osobna, ale ich integracja w spójny, działający system pozostaje zadaniem wymagającym praktycznego doświadczenia. Zamiarem tej pracy było zweryfikowanie na rzeczywistym przykładzie, jakie kompromisy i koszty niesie za sobą uruchomienie takiego systemu w chmurze publicznej. Udokumentowani

1.3 Przegląd technologii chmurowych w kontekście mikroserwisów

Chmura obliczeniowa definiowana jest jako model umożliwiający wygodny, sieciowy dostęp na żądanie do współdzielonej puli konfigurowalnych zasobów obliczeniowych, które mogą być szybko przydzielane i zwalniane przy minimalnym zaangażowaniu dostawcy [5]. Można w niej wyróżnić trzy modele usługowe. SaaS (*Software as a Service*), w którym produktem jest kompletne oprogramowanie dostępne przez sieć. IaaS (*Infrastructure as a Service*), w którym dostawca udostępnia surowe zasoby obliczeniowe, sieciowe i dyskowe. PaaS (*Platform as a Service*), w którym udostępniana jest gotowa platforma uruchomieniowa dla aplikacji. Usługi wykorzystane w tej pracy sytuują się pomiędzy modelami IaaS i PaaS. Dostawca przyjmuje odpowiedzialność utrzymania platformy uruchomieniowej. Pozostawiając użytkownikowi kontrolę nad jej konfiguracją. Z perspektywy architektury mikroserwisowej kluczowe znaczenie mają cztery grupy technologii.

Konteneryzacja pozwalająca opakować usługę wraz z jej zależnościami w przenośny niezmienny obraz, której standardem de facto stał się Kubernetes: platforma automatyzująca rozmieszczanie, skalowanie, samonaprawianie i aktualizacje kontenerów na klastrze maszyn [3]. Wszyscy najwięksi dostawcy chmury publicznej oferują zarządzane warianty Kubernetes. Różnice między największymi (Amazon EKS, Google GKE, Azure AKS) przedstawiono w rozdziale 4.

infrastructure as code (IaC), takie jak Terraform, pozwalające opisać całość infrastruktury chmurowej w postaci deklaratywnego, wersjonowanego kodu. Podejście to eliminuje ręczną, niepowtarzalną konfigurację środowisk i umożliwia ich odtworzenie w sposób w pełni zautomatyzowany. Ostatnią grupą są narzędzia wspierające eksploatację systemów rozproszonych. Takie jak systemy monitoringu i obserwowalności. Brokery komunikatów umożliwiające asynchroniczną wymianę zdarzeń. CI/CD automatyzujące zmiany w repozytorium i środowiku produkcyjnym.

1.4 Struktura pracy

Praca składa się z jedenastu rozdziałów, które można podzielić na część teoretyczną (rozdziały 2–4), projektową (rozdział 5), implementacyjno-wdrożeniową (rozdziały 6–9) oraz badawczo-podsumowującą (rozdziały 10–11).

Rozdział 2 przedstawia teoretyczne podstawy architektury mikroserwisowej: porównanie z architekturą monolityczną, modele komunikacji synchronicznej i asynchronicznej oraz wzorce projektowe stosowane w systemach rozproszonych. Rozdział 3 omawia platformę Kubernetes: jej podstawowe obiekty, mechanizmy skalowania, trwałego składowania danych, obserwowalności i bezpieczeństwa. Rozdział 4 charakteryzuje chmurę Amazon Web Services oraz usługi wykorzystane w projekcie, wraz z porównaniem Amazon EKS z konkurencyjnymi zarządzanymi usługami Kubernetes.

Rozdział 5 zawiera projekt systemu obsługi zamówień: opis funkcjonalny, model domenowy, architekturę usług, projekt interfejsów REST i kontraktów zdarzeń, a także uzasadnienie wyboru technologii implementacyjnych. Rozdziały 6 i 7 opisują implementację odpowiednio mikroserwisów backendowych oraz webowego portalu obsługi zamówień. Rozdział 8 przedstawia budowę kompletnego środowiska chmurowego w podejściu *infrastructure as code*.

Rozdział 9 poświęcony jest testom systemu oraz analizie jego działania w środowisku docelowym, w tym analizie kosztów wykorzystania usług chmurowych i identyfikacji wąskich gardeł. Pracę zamyka rozdział 10, zawierający ocenę stopnia realizacji postawionych celów, omówienie napotkanych trudności oraz rekomendacje dotyczące dalszego rozwoju systemu.

2. Teoretyczne podstawy architektury mikroserwisowej

Rozdział ten przedstawia teoretyczne podstawy architektury mikroserwisowej. Omówiono w nim różnice pomiędzy architekturą monolityczną a mikroserwisową, kluczowe cechy mikroserwisów, modele komunikacji międzyusługowej, a także wzorce projektowe stosowane w systemach rozproszonych. Rozdział zamyka omówienie konteneryzacji oraz roli orkiestracji, stanowiące wprowadzenie do rozdziału 3.

2.1 Architektura monolityczna a mikroserwisowa

Architektura monolityczna to model budowy aplikacji, w której całość logiki biznesowej kompiluje się i wdraża jako pojedyncza jednostka uruchomieniowa. Model ten ma istotne zalety. Jest on prosty koncepcyjnie. Transakcje mogą być lokalne i obejmować cały stan aplikacji. Uproszczone jest testowanie i debugowanie, a wdrożenie sprowadza się do uruchomienia jednego artefaktu. Z tych powodów monolit pozostaje racjonalnym wyborem dla małych zespołów i systemów o ograniczonej złożoności [1].

Problemy architektury monolitycznej ujawniają się wraz ze wzrostem systemu i organizacji. Skalowanie możliwe jest poprzez dodanie zasobów lub powielenie całej aplikacji. Nawet jeśli wąskim gardłem jest pojedynczy moduł. Każda zmiana wymaga zbudowania i wdrożenia całej aplikacji. Co wydłuża cykl wydawniczy i zwiększa ryzyko regresji. Brak wymuszenia granic pomiędzy modułami może powodować z czasem erozję, prowadząc do silnego sprzężenia kodu. Finalnie cały system związany jest jedną technologią i jedną wersją zależności, a awaria dowolnego fragmentu może unieruchomić aplikację w całości [2].

Architektura mikroserwisowa stanowi odpowiedź na te ograniczenia. Lewis i Fowler definiują ją jako podejście, w którym pojedyncza aplikacja budowana jest jako zbiór niewielkich usług. Każda działa we własnym procesie i komunikuje się z pozostałymi za pomocą lekkich mechanizmów. Usługi zbudowane są wokół zdolności biznesowych i wdrażane niezależnie w sposób w pełni zautomatyzowany [6]. Zestawienie najważniejszych różnic pomiędzy oboma podejściami przedstawia tabela 2.1.

Tabela 2.1: Porównanie architektury monolitycznej i mikroserwisowej (opracowanie własne na podstawie [1], [2])

Kryterium	Architektura monolityczna	Architektura mikroserwisowa
Jednostka wdrożenia	Cała aplikacja wdrażana jako jedna całość	Pojedyncza, niezależna usługa
Skalowanie	Skalowanie całej aplikacji, nawet gdy obciążony jest tylko jeden moduł	Skalowanie niezależne, dostosowane do potrzeb konkretnej usługi
Baza danych	Najczęściej jedna wspólna baza danych	Odrębna baza danych lub schemat danych dla każdej usługi
Komunikacja wewnętrzna	Bezpośrednie wywołania funkcji lub metod w ramach jednej aplikacji	Komunikacja sieciowa, np. REST, komunikaty lub zdarzenia
Spójność danych	Zwykle zapewniana przez transakcje ACID	Często oparta na spójności ostatecznej
Dobór technologii	Zazwyczaj jednolity stos technologiczny	Możliwość niezależnego doboru technologii dla poszczególnych usług
Izolacja awarii	Ograniczona; awaria jednego modułu może wpływać na całą aplikację	Większa; awaria może zostać ograniczona do pojedynczej usługi
Złożoność operacyjna	Relatywnie niska, prostsze wdrażanie i monitorowanie	Wysoka, wymaga orkiestracji, monitoringu i zarządzania komunikacją
Cykl wydawniczy	Wspólny dla całej aplikacji, zwykle mniej elastyczny	Niezależny dla poszczególnych usług, umożliwiający częstsze wdrożenia

Należy podkreślić, że mikroserwisy nie są rozwiązaniem uniwersalnie lepszym — przenoszą one złożoność z wnętrza aplikacji do przestrzeni pomiędzy usługami: do sieci, infrastruktury i procesów wdrożeniowych. Wybór tej architektury jest uzasadniony wówczas, gdy korzyści z niezależnego skalowania i wdrażania przewyższają koszt zwiększonej złożoności operacyjnej; właśnie dlatego praktyczna realizacja mikroserwisów jest dziś nierozdzielnie związana z technologiami chmurowymi i automatyzacją, które ten koszt istotnie redukują.

Należy podkreślić, że mikroserwisy nie są rozwiązaniem uniwersalnie lepszym. Przenoszą one złożoność z wnętrza aplikacji do przestrzeni pomiędzy usługami. Takimi jak sieć, infrastruktura czy procesy wdrożeniowe. Wybór tej architektury jest uzasadniony wówczas, gdy korzyści z niezależnego skalowania i wdrażania przewyższają koszt zwiększonej złożoności operacyjnej. Z tych powodów praktyczna realizacja mikroserwisów jest bardzo związana

z technologiami chmurowymi i ich automatyzacją.

2.2 Charakterystyka mikroservisów

Literatura przedmiotu [1], [2] wskazuje zestaw cech konstytuujących architekturę mikroservisową, które można sprowadzić do następujących zasad.

Modelowanie wokół domeny biznesowej. Granice usług powinny odpowiadać granicom kontekstów biznesowych (*bounded contexts*) w rozumieniu projektowania sterowanego domeną (*Domain-Driven Design*) [7]. Usługa bierze odpowiedzialność za wyodrębniony fragment domeny. W realizowanym systemie ten podział widać na: zamówieniach, katalog produktów raz płatnościami. Minimalizuje on liczbę zmian wymagających na koordynacji między usługami.

Niezależność wdrożeniowa. Każda usługa może być zbudowana, przetestowana i wdrożona bez udziału pozostałych. Separacja ta pozwala rozwijać i testować aplikacje samodzielnie. Wymaga to jawnych wersjonowanych kontraktów komunikacyjnych oraz unikanie współdzielenia struktur danych. Bardzo dobrą praktyką jest rozdzielenie struktur na komunikacyjną i transportową.

Ukrywanie informacji i własność danych. Usługa udostępnia na zewnątrz wyłącznie swój interfejs, ukrywając szczegóły implementacji. Konsekwencją jest zasada *database-per-service* każda usługa jest wyłącznym właścicielem swoich danych. Inne usługi mogą uzyskać do nich dostęp jedynie poprzez jej interfejs. Złamanie tej zasady prowadzi do sprzężenia na poziomie schematu, które w praktyce odbiera usługom niezależność wdrożeniową [1].

Decentralizacja i autonomia technologiczna. Usługi mogą być rozwijane przez niezależne zespoły i realizowane w różnych technologiach, dobranych do charakteru problemu.

Odporność na awarie. W systemie rozproszonym awarie są normalnym zjawiskiem. Architektura musi zakładać że usługi mogą być chwilowo niedostępne. Ograniczenie takich awarii można osiągnąć poprzez asynchroniczną komunikację oraz samonaprawiające się usługi.

Obserwowalność. Zachowanie systemu rozproszonego wymaga dokładnego monitorowania. Z powodu że każda usługa działa całkowicie oddzielnie. Wymagane jest dokładne monitorowanie każdej z nich w celu wykrywania anomalii. Niezbędne więc staje się gromadzenie metryk, logów i informacji o stanie każdej usługi. Zagadnienie to rozwinięto w rozdziale 3 oraz, w odniesieniu do implementacji, w rozdziale 6.

2.3 Komunikacja synchroniczna i asynchroniczna

Dekompozycja systemu na usługi czyni z komunikacji międzyusługowej zagadnienie architektonicznie pierwszorzędne. Wyróżnia się dwa zasadnicze modele tej komunikacji, różniące się sprzężeniem czasowym pomiędzy stronami.

2.3.1 Komunikacja synchroniczna — REST

W modelu synchronicznym usługa wywołująca wysyła żądanie i wstrzymuje przetwarzanie do czasu otrzymania odpowiedzi. Najpowszechniejszą realizacją tego modelu są interfejsy REST (*Representational State Transfer*) oparte na protokole HTTP i formacie JSON: zasoby identyfikowane są adresami URL, a operacje na nich wyrażane metodami HTTP (GET, POST, PUT, DELETE). Zaletami podejścia są prostota, powszechna znajomość, łatwość testowania oraz naturalna zgodność z modelem żądanie–odpowieź, właściwym dla interakcji inicjowanych przez użytkownika.

W modelu synchronicznym usługa wywołująca wysyła żądanie i wstrzymuje przetwarzanie do czasu otrzymania odpowiedzi. Najpowszechniejszą realizacją tego modelu są interfejsy REST (*Representational State Transfer*) oparte na protokole HTTP i formacie JSON. Zasoby identyfikowane są za pomocą adresów URL, a operacje metodami HTTP (GET, POST, PUT, DELETE). Zaletami podejścia jest prostota, łatwość testowania oraz zgodność z modelem żądanie–odpowieź. Podobnym dla interakcji użytkownika z serwisem.

Wadą komunikacji synchronicznej jest sprzężenie czasowe. Obie strony muszą być dostępne jednocześnie, a niedostępność lub spowolnienie usługi wywoływanej propaguje się na wywołującą. W łańcuchach wywołań prowadzi to do awarii kaskadowych, w których problem jednej usługi unieruchamia kolejne [2]. Powoduje to złamanie paradygmatu mikroserwisów czyli odporność na awarie. Z tego względu komunikacje synchroniczną stosuje się głównie na styku użytkownik-system.

2.3.2 Komunikacja asynchroniczna i architektura sterowana zdarzeniami

W modelu asynchronicznym nadawca przekazuje komunikat pośrednikowi brokerowi komunikatów i natychmiast kontynuuje przetwarzanie, nie oczekując na odbiorców. Szczególną postacią tego modelu jest architektura sterowana zdarzeniami (*event-driven architecture*), w której usługi publikują zdarzenia domenowe. Niemutowalne fakty opisujące to, co zaszło w ich obszarze odpowiedzialności (np. *utworzono zamówienie*). Subskrybenci pobierają te zdarzenia i obsługują je według własnej logiki. Dzięki temu rozszczepieniu serwisów, nadawca nie zna swoich odbiorców. Powoduje to łatwe dodawanie kolejnych subskrybentów bez zmian po stro-

nie producenta [8].

Model ten eliminuje sprzężenie czasowe: chwilowa niedostępność konsumenta nie wpływa na producenta, a zaległe komunikaty zostaną przetworzone po przywróceniu jego działania. Ceną jest rezygnacja z natychmiastowej, silnej spójności danych na rzecz spójności ostatecznej (*eventual consistency*) — stan rozproszony pomiędzy usługami uzgadnia się z opóźnieniem, co projekt systemu musi jawnie uwzględniać. Istotne są również gwarancje dostarczania: brokery komunikatów typowo zapewniają dostarczenie *co najmniej raz* (*at-least-once*), co oznacza, że ten sam komunikat może zostać doręczony wielokrotnie — konsumenci muszą zatem przetwarzać komunikaty w sposób idempotentny [8].

Model ten eliminuje sprzężenie czasowe. Chwilowa niedostępność konsumenta nie wpływa na producenta. Wiadomości nieprzetworzone będą czekały w systemie kolejkowym do ponownego działania subskrybenta. Ceną jest rezygnacja z natychmiastowej, silnej spójności danych na rzecz spójności ostatecznej (*eventual consistency*). Stan rozproszony pomiędzy usługami może być obsługiwany z opóźnieniem, co system musi uwzględniać w finalnej architekturze. Istotne są również gwarancje dostarczania. Brokery komunikatów typowo zapewniają dostarczenie *co najmniej raz*, co oznacza, że ten sam komunikat może zostać doręczony wielokrotnie. Konsumenci muszą zatem przetwarzać komunikaty w sposób idempotentny [8].

W zrealizowanym systemie oba modele zastosowano komplementarnie: żądania użytkownika obsługiwane są synchronicznie przez REST. Natomiast proces walidacji zamówienia i rezerwacji stanów magazynowych przebiega asynchronicznie, poprzez zdarzenia publikowane do brokera Apache Kafka. Szczegóły tego podziału omówiono w rozdziale 5.

2.4 Wzorce projektowe w systemach mikroserwisowych

Problemy powtarzające się w systemach mikroserwisowych doczekały się skatalogowanych rozwiązań — wzorców projektowych [2]. Poniżej omówiono wzorce, które znalazły bezpośrednie zastosowanie w zrealizowanym systemie; odwołania do ich implementacji zawiera rozdział 6.

Baza danych na usługę (*database-per-service*). Każda usługa posiada wyłączność na swoje dane i udostępnia je jedynie poprzez interfejs. Wzorec ten został zaimplementowany tylko częściowo gdzie serwis zamówień nie dotyczy danych produktowych i płatniczych. Wysyłając asynchronicznie zapytanie o rezerwację produktów.

Zdarzenia domenowe i choreografia. Proces biznesowy obejmujący wiele usług może być koordynowany centralnie (orkiestracja) lub realizować się jako łańcuch reakcji na zdarzenia (choreografia). Każda usługa, przetworzywszy swój etap, publikuje zdarzenie, na które reagują

kolejne. W zrealizowanym systemie cykl walidacji zamówienia przebiega według choreografii. Serwis zamówień publikuje zdarzenie utworzenia zamówienia, na które serwis produktów odpowiada rezerwacją stanów magazynowych i publikacją wyniku walidacji.

Idempotentny odbiorca i klucz idempotencji. Komunikaty mogą docierać wielokrotnie, a klient może ponowić żądanie (np. po utracie odpowiedzi w sieci). Operacje zmieniające stan muszą być odporne na powtórzenia. Realizuje się to poprzez klucz idempotencji. Jest to unikalny identyfikator operacji przekazywany przez klienta i zapamiętywany przez usługę. Dzięki tej logice powtórne żądanie nie tworzy duplikatu, a zwraca wynik już wykonanej operacji.

Maszyna stanów. Cykl życia encji biznesowej w omawianym systemie zamówienia modelowany jest jako skończony automat stanów. Z jawnie zdefiniowanym zbiorem dozwolonych przejść. Wzorzec ten uniemożliwia wprowadzenie encji w stan niespójny (np. opłacenie zamówienia anulowanego) i porządkuje logikę biznesową wokół przejść pomiędzy stanami.

Brama API i odwrócone proxy. Klient systemu nie komunikuje się z usługami bezpośrednio. Istnieje pojedynczy punkt wejścia który przekierowuje żądania do odpowiednich serwisów na podstawie ścieżki. Upraszcza to konfiguracje klientów. Pozwala uniknąć CORS. Ukrywa wewnętrzną topologię systemu.

Konfiguracja zewnętrzna. Zgodnie z metodyką *The Twelve-Factor App* konfiguracja zależna od środowiska (adresy usług, poświadczenia, przełączniki funkcji) przechowywana jest poza kodem. W zmiennych środowiskowych, dzięki czemu ten sam artefakt może być wdrażany w różnych środowiskach bez rekompilacji [9].

Kontrola zdrowia (*health check*). Każda usługa udostępnia punkt końcowy raportujący jej stan. Odpytywany jest przez platformę uruchomieniową w celu wykrywania niezdrowych instancji. Zapiega to awarii poprzez automatyczny ich restart lub wyłączenie z puli.

2.5 Konteneryzacja i rola orkiestracji

Praktyczna realizacja architektury mikroservisowej wymaga jednolitego sposobu pakowania i uruchamiania wielu niezależnych usług. Rolę tę pełni konteneryzacja. Jest to technika wirtualizacji na poziomie systemu operacyjnego. Procesy izolowane są od siebie przy użyciu jądra Linux, współdzieląc przy tym jądro systemu gospodarza. W odróżnieniu od maszyn wirtualnych kontenery nie wymagają odrębnego systemu operacyjnego, dzięki czemu uruchamiają się w ułamkach sekund i wprowadzają znikomy narzut wydajnościowy [3].

Jednostką dystrybucji jest obraz kontenera. Niemutowalny, warstwowy artefakt zawierający aplikację wraz z kompletem jej zależności, budowany na podstawie deklaratywnego opisu (*Dockerfile*). Przechowywany jest on w rejestrze obrazów. Niemutowalność obrazu zapewnia

powtarzalność wdrożeń. Eliminuje to błędy wynikające z różnic z środowiskami.

Konteneryzacja rozwiązuje problem pojedynczej usługi. Jednak nie odpowiada na pytania jak: Na którym węźle uruchomić kontener? Jak utrzymać zadaną liczbę jego replik mimo awarii węzłów? Jak kierować ruch do zmieniających się dynamicznie instancji? Jak przeprowadzić aktualizację bez przerwy w działaniu? Jak skalować liczbę replik w odpowiedzi na obciążenie. Zadania te są określane jako orkiestracja kontenerów. Wymaga to specjalnej platformy której standardem stał się Kubernetes, któremu poświęcono następny rozdział.

3. Kubernetes jako platforma uruchomieniowa mikroserwisów

Kubernetes jest otwartoźródłową platformą orkiestracji kontenerów, automatyzującą ich rozmieszczanie, skalowanie i eksploatację na klastrze maszyn. Projekt, zainicjowany przez firmę Google w 2014 roku i czerpiący z jej wieloletnich doświadczeń z wewnętrznym systemem Borg, został przekazany fundacji Cloud Native Computing Foundation i stał się standardem przemysłowym w obszarze orkiestracji [3].

U podstaw działania Kubernetes leży model deklaratywny. Użytkownik zamiast wydawać platformie polecenia, opisuje pożądany stan systemu (np. *trzy repliki usługi zamówień w wersji 1.2*). Platforma ciągle porównuje stan faktyczny z pożądanym. Przy różnicach niweluje rozbieżności (pętla uzgadniania, *reconciliation loop*). Z takiego modelu wynika samonaprawialność systemu. Awaria jakiegoś kontenera, węzła czy usługi powoduje rozbieżność którą platforma próbuje usunąć.

Architektura klastra Kubernetes obejmuje płaszczyznę sterowania (*control plane*), serwer API, magazyn stanu etcd, planistę (*scheduler*) oraz menedżery kontrolerów. Obejmuje również węzły robocze (*worker nodes*), na których agent *kubelet* uruchamia kontenery zgodnie z decyzjami planisty [10]. Taka architektura pozwala usługom chmurowym wziąć odpowiedzialność za płaszczyznę sterowania. Dzięki czemu użytkownik zarządza wyłącznie węzłami roboczymi i uruchamianymi obciążeniami (rozdział 4).

3.1 Podstawowe obiekty Kubernetes

Stan klastra opisywany jest za pomocą obiektów API. Poniżej omówiono obiekty wykorzystane w zrealizowanym systemie; ich konkretne definicje przedstawiono w rozdziale 8.

Pod jest najmniejszą jednostką wdrożeniową. Grupą jednego lub kilku kontenerów współdzielących przestrzeń sieciową (wspólny adres IP) i woluminy. Najczęściej w praktyce zwiera jeden kontener aplikacyjny. Pody są ulotne, czyli w każdej chwili mogą zostać usunięte i utworzone w innym miejscu (węźle). Dzieje się tak np podczas balansowania gdzie Kubernetes

próbuję zmieścić nowe Pody podczas wdrożenia. Wiąże się to z tym że jeśli aplikacja wymaga utrzymania stanu to musi ją trzymać w zewnętrznym miejscu jak baza danych.

Deployment deklaruje pożądaną liczbę replik podając wraz z szablonem jego specyfikacji i strategią aktualizacji. Za pośrednictwem obiektu `ReplicaSet` utrzymuje zadaną liczbę replik. Przy zmianie szablonu (np. nowej wersji obrazu) przeprowadza aktualizację kroczącą (*rolling update*). Nowe repliki uruchamiane są stopniowo, równolegle z wygaszaniem starych. Pozwala to wdrażać zmiany bez przerwy w dostępności usługi. W zrealizowanym systemie każdy z trzech mikroserwisów opisany jest własnym `Deploymentem`.

Service rozwiązuje problem ulotności podów, udostępniając stabilny, wirtualny punkt dostępu (nazwę DNS i adres IP) dla dynamicznie zmieniającego się zbioru replik, wraz z równoważeniem obciążenia pomiędzy nimi. Wariant `ClusterIP` udostępnia usługę wewnątrz klastra, `NodePort` — na stałym porcie każdego węzła (wariant ten wykorzystano do wystawienia ruchu z zewnętrznego load balancera), a `LoadBalancer` zleca utworzenie zewnętrznego balancera dostawcy chmury.

Namespace dzieli klastry na logiczne przestrzenie nazw, izolując zasoby różnych zespołów lub systemów i stanowiąc granicę dla uprawnień oraz limitów zasobów. Komponenty zrealizowanego systemu działają w dedykowanej przestrzeni `order-portal`.

ConfigMap i Secret przechowują konfigurację oddzieloną od obrazów kontenerów. `ConfigMap` zawiera jawną, a `secret` poufną. Udostępniają one podom zmienne środowiskowe lub pliki. Obiekty te są infrastrukturalną realizacją wzorca konfiguracji zewnętrznej (sekcja 2.4).

3.2 Zarządzanie konfiguracją i skalowaniem

Specyfikacja każdego kontenera może określać zapotrzebowanie na zasoby w postaci żądań (*requests*) i limitów (*limits*) dla procesora i pamięci. Żądania stanowią podstawę decyzji planisty — pod zostanie umieszczony wyłącznie na węźle dysponującym wolnymi zasobami w zadeklarowanej wielkości — natomiast limity wyznaczają górną granicę zużycia, której przekroczenie skutkuje dławieniem procesora lub zakończeniem procesu w przypadku pamięci. Poprawne określenie tych wartości ma szczególne znaczenie na węzłach o małej pojemności, jakie wykorzystano w środowisku zrealizowanego systemu.

W specyfikacji każdego kontenera należy określić zapotrzebowanie na zasoby. Występują one w postaci żądań (*requests*) i limitów (*limits*) dla procesora i pamięci. Na podstawie żądań planista rozmieszcza pody w węzłach dysponujących wolnymi zasobami w zadeklarowanej wielkości. Limity natomiast wyznaczają górną granicę zużycia, której przekroczenie powoduje dławieniem procesora lub zakończeniem procesu w przypadku pamięci. Poprawne określenie

tych wartości ma szczególne znaczenie na węzłach o małej pojemności, jakie wykorzystano w środowisku zrealizowanego systemu.

Konfiguracja aplikacji dostarczana jest podobnie ze źródeł zewnętrznych. Obiektów ConfigMap i Secret. W zrealizowanym systemie poświadczenia bazy danych trafiają do podów jako zmienne środowiskowe z obiektów Secret, zasilanych z usługi AWS Secrets Manager (rozdział 8).

Skalowanie obciążeń realizowane jest na dwóch poziomach. Poziomy pierwszy stanowi **Horizontal Pod Autoscaler** (HPA). Kontroler, który cyklicznie porównuje zmierzone wykorzystanie zasobów (domyślnie procesora, na podstawie danych z komponentu *metrics-server*). Z wartością docelową i odpowiednio zwiększa lub zmniejsza liczbę replik Deploymentu w zadanym przedziale [10]. Poziom drugi to skalowanie samego klastra, czyli liczby węzłów roboczych.

3.3 Trwale składowanie danych

System plików kontenera jest ulotny — ginie wraz z jego restartem — a pody mogą być przenoszone pomiędzy węzłami, co czyni przechowywanie danych na dysku węzła zawodnym. Kubernetes rozwiązuje ten problem warstwą abstrakcji nad pamięcią trwałą, rozdzielającą perspektywę administratora i aplikacji [10].

Zgodnie z definicją Poda, system plików kontenera jest ulotny. Pody mogą być przenoszone między węzłami. Z tego powodu Kubernetes rozwiązuje ten problem warstwą abstrakcji nad pamięcią trwałą, rozdzielającą perspektywę administratora i aplikacji [10].

PersistentVolume (PV) reprezentuje fizyczny zasób pamięci — np. wolumin blokowy dostawcy chmury — natomiast **PersistentVolumeClaim** (PVC) jest żądaniem aplikacji: deklaracją potrzebnej pojemności i trybu dostępu, niezależną od tego, skąd zasób pochodzi. **StorageClass** opisuje klasę pamięci (typ nośnika, parametry wydajnościowe) i umożliwia provisionowanie dynamiczne: wolumin tworzony jest automatycznie w chwili pojawienia się żądania, bez ręcznego przygotowywania zasobów przez administratora. Tworzeniem i dołączaniem woluminów zarządza sterownik zgodny ze standardem CSI (*Container Storage Interface*), specyficzny dla danego dostawcy infrastruktury.

PersistentVolume (PV) reprezentuje fizyczny zasób pamięci. Natomiast **PersistentVolumeClaim** (PVC) jest żądaniem zapotrzebowania aplikacja na taki wolumin o potrzebnej pojemności i trybie dostępu. **StorageClass** opisuje klasę pamięci (typ nośnika, parametry wydajnościowe) i umożliwia provisionowanie dynamiczne. Wolumin tworzony jest automatycznie w chwili pojawienia się żądania. Tworzeniem i dołączaniem woluminów zarządza sterownik zgodny ze

standardem CSI (*Container Storage Interface*), specyficzny dla danego dostawcy infrastruktury.

W zrealizowanym systemie pamięci trwałej wymagają broker Kafka oraz serwer Prometheus. Wykorzystano sterownik AWS EBS CSI z klasą pamięci opartą na woluminach gp3, skonfigurowaną z trybem wiązania *WaitForFirstConsumer*. Wolumin tworzony jest dopiero po zaplanowaniu poda na konkretnym węźle, co gwarantuje jego utworzenie w tej samej strefie dostępności, w której działa pod. Główna baza danych systemu pozostaje natomiast poza klastrem, jako zarządzana usługa Amazon RDS. Uzasadnienie tej decyzji przedstawiono w rozdziale 4.

3.4 Obserwowalność: Prometheus i Grafana

Obserwowalność systemu rozproszonego opiera się na trzech filarach: metrykach (liczbowych szeregach czasowych opisujących stan systemu), logach (zapisach zdarzeń) oraz śladach rozproszonych (zapisach przebiegu pojedynczych żądań przez kolejne usługi). W zrealizowanym systemie zastosowano dwa pierwsze filary, z metrykami gromadzonymi przez Prometheus [11].

Prometheus działa w modelu *pull*. Aplikacje wystawiają punkty końcowe z metrykami, które cyklicznie pobiera. Zbiór celów nie jest konfigurowany statycznie, a w środowisku Kubernetes Prometheus odkrywa je dynamicznie poprzez API klastra. O objęciu poda monitoringiem decydują jego adnotacje (`prometheus.io/scrape`, `prometheus.io/port`). ozwiązanie to wykorzystano w zrealizowanym systemie: każdy mikroserwis opatrzony jest adnotacjami wskazującymi port techniczny, na którym udostępnia metryki.

Metryki przyjmują postać nazwanych szeregów czasowych z etykietami (*labels*), umożliwiającymi agregację i filtrowanie np. liczba żądań HTTP w podziale na ścieżkę i kod odpowiedzi. Typowe rodzaje metryk to liczniki (*counters*), mierniki (*gauges*) oraz histogramy, pozwalające szacować percentyle czasów odpowiedzi. Zapytania do zgromadzonych danych formułowane są w języku PromQL, który umożliwia m.in. wyznaczanie szybkości zmian liczników czy agregacje po etykietach.

Warstwę prezentacji stanowi Grafana — narzędzie wizualizacji, które na podstawie zapytań PromQL buduje interaktywne kokpity (*dashboards*) prezentujące stan systemu w czasie rzeczywistym i historycznie. W obu narzędziach konfigurację (źródła danych, definicje kokpitów) można dostarczać deklaratywnie, co w zrealizowanym systemie wykorzystano do provisionowania Grafany z poziomu kodu infrastruktury (rozdział 8). Zakres metryk publikowanych przez poszczególne mikroserwisy — w tym metryk domenowych, takich jak liczba utworzonych zamówień — omówiono w rozdziale 6.

Warstwę prezentacji stanowi Grafana — narzędzie wizualizacji, które na podstawie za-

pytań PromQL buduje interaktywne kokpity (*dashboards*) prezentujące stan systemu w czasie rzeczywistym i historycznie. Zakres metryk publikowanych przez poszczególne mikroserwisy omówiono w rozdziale 6.

3.5 Mechanizmy bezpieczeństwa w Kubernetes

Bezpieczeństwo klastra Kubernetes obejmuje kilka współdziałających warstw [10].

Uwierzytelnianie i autoryzacja dostępu do API. Każde żądanie do serwera API podlega uwierzytelnieniu, a następnie autoryzacji w modelu RBAC (*Role-Based Access Control*): role definiują zestawy dozwolonych operacji na zasobach, a powiązania ról (*role bindings*) przypisują je użytkownikom lub kontom usług. Procesy działające w klastrze występują wobec API jako **ServiceAccount** — konta usług o jawnie nadawanych, minimalnych uprawnieniach. W zrealizowanym systemie własne konto usługi z ograniczoną rolą posiada serwer Prometheus, któremu do dynamicznego odkrywania celów pomiarowych wystarczą uprawnienia tylko do odczytu metadanych podów.

Każde żądanie do serwera API podlega uwierzytelnieniu i autoryzacji w modelu RBAC (*Role-Based Access Control*). Role definiują zestawy dozwolonych operacji na zasobach. Powiązania ról (*role bindings*) przypisują je użytkownikom lub kontom usług. Procesy działające w klastrze występują wobec API jako **ServiceAccount**. Konta usług o jawnie nadawanych, minimalnych uprawnieniach. W zrealizowanym systemie własne konto usługi z ograniczoną rolą posiada serwer Prometheus. Posiada uprawnienia do dynamicznego odkrywania celów poprzez odczyt metadanych podów.

Ochrona danych poufnych. Obiekty Secret oddzielają poświadczenia od obrazów i manifestów aplikacji, a dostęp do nich podlega RBAC. W zrealizowanym systemie źródłem poświadczeń bazy danych jest AWS Secrets Manager.

Kontekst bezpieczeństwa podów. Specyfikacja *securityContext* pozwala wymusić uruchamianie procesów bez uprawnień administratora (*nonroot*). Komplementarną praktyką jest minimalizacja samych obrazów. Obrazy klasy *distroless* pozbawione są powłoki i menedżera pakietów, redukują powierzchnię ataku kontenera. Podejście to zastosowano dla wszystkich mikroserwisów systemu (rozdział 6).

Segmentacja sieci. Domyślnie komunikacja pomiędzy podami w klastrze jest nieograniczona. Obiekty NetworkPolicy pozwalają zawęzić ją do jawnie dozwolonych przepływów. W zrealizowanym systemie segmentację ruchu zrealizowano przede wszystkim na poziomie infrastruktury chmurowej. Grup bezpieczeństwa i topologii podsieci (rozdział 8).

Integracja z systemem uprawnień chmury. W środowiskach zarządzanych pody czę-

sto wymagają dostępu do usług dostawcy chmury (np. tworzenia woluminów EBS). Zamiast statycznych kluczy stosuje się mechanizmy wiążące konta usług Kubernetes z rolami IAM — w EKS są to IRSA (*IAM Roles for Service Accounts*) oraz nowszy mechanizm EKS Pod Identity, który w zrealizowanym systemie nadaje uprawnienia sterownikowi EBS CSI. Proces otrzymuje wówczas poświadczenia krótkoterminowe, automatycznie rotowane, o zakresie ograniczonym do przypisanej roli.

Integracja z systemem uprawnień chmury. W środowiskach zarządzanych poddy często wymagają dostępu do usług dostawcy chmury. Wiąże się wtedy konta usług Kubernetes z rolami dostawcy. W zrealizowanym systemie użyty został EKS Pod Identity, który nadaje uprawnienia sterownikowi EBS CSI.

Omówione mechanizmy stanowią fundament, na którym zbudowano środowisko uruchomieniowe systemu. Sposób ich praktycznego użycia przedstawiono w rozdziałach 4 i 8.

4. Amazon Web Services jako środowisko chmurowe

Amazon Web Services (AWS) jest największą pod względem udziału w rynku platformą chmury publicznej, oferującą kilkaset usług. Od surowych zasobów obliczeniowych, przez zarządzane bazy danych i platformy kontenerowe, po wyspecjalizowane usługi analityczne. Rozdział ten charakteryzuje platformę AWS oraz omawia usługi, z których zbudowano środowisko uruchomieniowe zrealizowanego systemu, a kończy się porównaniem Amazon EKS z konkurencyjnymi zarządzanymi usługami Kubernetes.

4.1 Charakterystyka AWS i jego architektury globalnej

Infrastruktura AWS jest podzielona **regiony**. Są to geograficznie odseparowane lokalizacje (np. `eu-central-1` we Frankfurcie). Każdy z nich jest oddzielną domeną awarii i osobnym miejscem przechowywania danych. Każdy region składa się z co najmniej trzech **stref dostępności** (*Availability Zones, AZ*). Są to fizycznie rozdzielone centra danych o niezależnym zasilaniu i łączności. Połączone są one ze sobą łączami o niskich opóźnieniach. Rozmieszczenie komponentów systemu w wielu strefach dostępności jest podstawową techniką zapewniania wysokiej dostępności. Awaria pojedynczej strefy nie przerywa działania systemu [12].

Model rozliczeń AWS ma wiele form. W podstawowej wersji opłaty naliczane są za faktyczne zużycie zasobów (czas działania instancji, pojemność, transfer danych). Bez zobowiązań wstępnych. W tym projekcie wykorzystana została pula bezpłatnych zasobów (*Free Tier*), udostępniona czasowo dla nowych kont. Z tego powodu wiele decyzji co do doboru parametrów środowiska została podjęta w zakresie bezpłatnego poziomu.

Istotnym elementem charakterystyki platformy jest **model odpowiedzialności współdzielonej** (*shared responsibility model*). AWS odpowiada za bezpieczeństwo *samej chmury*. Czyli infrastruktury fizycznej, hipernadzorcy, oprogramowania usług zarządzanych. Natomiast klient za bezpieczeństwo *w chmurze*. Czyli konfigurację sieci i uprawnień, ochronę danych oraz poprawność własnych aplikacji [12].

4.2 Amazon EKS — zarządzana usługa Kubernetes

Amazon Elastic Kubernetes Service (EKS) udostępnia klaster Kubernetes. Płaszczyzna sterowania jest w całości zarządzana przez AWS. Użytkownik widzi tylko punkt końcowy API [12]. Użytkownik odpowiada za dobór węzłów roboczych oraz uruchomienie obciążenia. EKS wykorzystuje niezmodyfikowaną, certyfikowaną dystrybucję Kubernetes, dzięki czemu obciążenia i manifesty pozostają przenośne pomiędzy środowiskami.

Warstwę obliczeniową klastra EKS można zrealizować na dwa sposoby. Jako samodzielnie zarządzane instancje EC2 **zarządzane grupy węzłów** (*managed node groups*). W których AWS automatyzuje provisionowanie, aktualizacje i wymianę instancji EC2. AWS Fargate, model bezserwerowy, w którym pojedyncze pody uruchamiane są bez jawnie widocznych węzłów. W zrealizowanym systemie wybrano zarządzane grupy węzłów: w odróżnieniu od Fargate zachowują one pełną kontrolę nad typem instancji.

Integralną częścią EKS są **dodatki** (*add-ons*). Zarządzane przez AWS komponenty systemowe klastra. Przykładem takiej wtyczki może być sieciowa VPC CNI. Przydziela ona podom adresy IP bezpośrednio z puli adresowej sieci VPC. Bez niej liczba podów na węźle jest ograniczona liczbą interfejsów sieciowych instancji. W tym projekcie użyłem małych instancji co stanowiłoby barierę przy wdrożeniu wszystkich komponentów.

Tożsamość i dostęp do klastra EKS realizowane są poprzez integrację z systemem IAM. Zarówno administratorzy, jak i węzły robocze uwierzytelniają się wobec API Kubernetes przy użyciu tożsamości IAM (sekcja 3.5).

4.3 Usługi powiązane wykorzystane w projekcie

4.3.1 Amazon EC2 jako warstwa obliczeniowa

Amazon Elastic Compute Cloud (EC2) dostarcza maszyny wirtualne (*instancje*). Mają one zróżnicowane zasoby opisane typami instancji. W zrealizowanym systemie użyte zostały jako węzły EKS. Stanowią one warstwę obliczeniową klastra. W celu zmieszczenia w darmowym programie użyte zostały instancje `t3.small` w trybie *on-demand*. Grupa węzłów posiada konfigurowalny przedział wielkości (minimalna, pożądana i maksymalna liczba instancji). Stanowi to infrastrukturalny fundament skalowania klastra.

4.3.2 Amazon RDS for PostgreSQL jako warstwa danych

Amazon Relational Database Service (RDS) jest zarządzaną usługą relacyjnych baz danych, obejmującą od użytkownika instalację, łatanie, kopie zapasowe i odtwarzanie silnika bazodanowego. W realizowanym systemie użyto RDS PostgreSQL. Korzystają z niej wszystkie 3 mikroserwisy. Instancja umieszczona jest w podsieci prywatnej. Z grupą bezpieczeństwa, która umożliwia dostęp tylko z sieci klastra. Decyzja użycia RDS zamiast bazy uruchamianej w klastrze, wynika z dostarczenia wszystkich potrzebnych mechanizmów dla bazy produkcyjnej przez serwis.

4.3.3 Amazon ECR — rejestr obrazów kontenerów

Amazon Elastic Container Registry (ECR) jest prywatnym rejestrem obrazów kontenerów, zintegrowanym z systemem uprawnień IAM. W zrealizowanym systemie utworzono po jednym repozytorium dla każdego mikroserwisu, z włączonym automatycznym skanowaniem obrazów pod kątem znanych podatności oraz polityką cyklu życia ograniczającą liczbę przechowywanych wersji obrazu (koszt składowania rośnie z każdą wersją, a starsze obrazy szybko tracą użyteczność). Węzły klastra pobierają obrazy na mocy roli IAM z uprawnieniem tylko do odczytu, bez jakichkolwiek statycznych poświadczeń.

Amazon Elastic Container Registry (ECR) jest prywatnym rejestrem obrazów kontenerów. W systemie utworzono repozytorium dla serwisu z skanowaniem obrazów pod kątem podatności. Ustawiono również cykl życia ograniczającą liczbę przechowywanych wersji obrazów. System ten zintegrowany jest z IAM. EKS za pomocą roli tylko do odczytu pobiera podczas deploymentu ostatnią wersję obrazu.

4.3.4 Amazon Cognito — uwierzytelnianie użytkowników

Amazon Cognito jest zarządzaną usługą tożsamości użytkowników końcowych. Jej centralnym pojęciem jest *pula użytkowników* (*user pool*). Jest to katalog kont wraz z mechanizmem rejestracji, logowania, polityki haseł i weryfikacji adresów e-mail. Po poprawnym uwierzytelnieniu wydaje podpisane cyfrowo tokeny JWT. Za ich pomocą serwisy mogą obsłużyć logowanie. Każdy token może być zweryfikowany na podstawie publicznych kluczy udostępnianych przez usługę. Serwis udostępnia również interfejs logowania (*Hosted UI*). Dzięki niemu aplikacja webowa nie musi mieć formularza do logowania tylko korzysta z udostępnionego przez AWS. Szczegóły projektu uwierzytelniania przedstawiono w rozdziale 5.

4.3.5 Amazon S3 i CloudFront — hosting aplikacji webowej

Amazon Simple Storage Service (S3) jest obiektywnym magazynem danych o praktycznie nieograniczonej pojemności. Amazon CloudFront jest siecią dystrybucji treści (CDN) serwującą zawartość z lokalizacji brzegowych położonych blisko użytkowników. Połączenie obu usług stanowi wzorcową architekturę hostingu aplikacji jednostronicowych (SPA). Zbudowane statyczne artefakty przechowywane są w S3, a CloudFront udostępnia je użytkownikom. Bucket w którym przechowywana jest strona jest prywatny. Z tego powodu publiczny dostęp jest tylko z serwisu CloudFront. Dostęp do niego CDN ma przez mechanizm *emphOrigin Access Control* (OAC). Cloudfront pełni w systemie również rolę pojedynczego punktu wejścia dla API który kieruje ruch do load balancera. Cloudfront cacheuje również statyczne dane na lokalizacjach brzegowych, więc ogranicza on zapytania do S3 i przyspiesza dostarczenia strony. Konfigurację tę omówiono w rozdziale 8.

4.3.6 Amazon VPC, ALB i architektura sieciowa

Amazon Virtual Private Cloud (VPC) udostępnia logicznie izolowaną sieć wirtualną o adresacji definiowanej przez użytkownika, dzieloną na podsieci rozmieszczone w strefach dostępności. Podstawowym wzorcem zastosowanym w zrealizowanym systemie jest podział na podsieci publiczne i prywatne. Sieć publiczna ma dostęp do internetu dzięki temu może dostać ruch sieciowy. Sieć prywatna za to go nie ma i ruch do niej przechodzi przez bramę NAT. Z tego powodu generalną praktyką jest wysyłanie ruchu internetowego do sieci publicznej i z sieci publicznej przez NAT do prywatnej. Ruch sieciowy filtrowany jest przez **grupy bezpieczeństwa** (*security groups*). Są to tanowe zapory przypisywane zasobom, zezwalające wyłącznie na jawnie zdefiniowane przepływy.

Application Load Balancer (ALB) jest balancerem warstwy siódmej, trasującym żądania HTTP do zarejestrowanych celów i weryfikującym ich zdrowie. W zrealizowanym systemie ALB przyjmuje ruch API od CloudFront i kieruje go do węzłów klastra. Dostęp do balancera ograniczono do zakresów adresowych CloudFront, a żądania pozbawione ustawianego przez CloudFront nagłówka weryfikacyjnego są odrzucane. Wymusza przepływ całego ruchu przez warstwę brzegową.

4.3.7 IAM i Secrets Manager

AWS Identity and Access Management (IAM) jest systemem zarządzania tożsamością i uprawnieniami obejmującym całą platformę. Uprawnienia działają na zasadzie polityk przypisywanych użytkownikom i i **rolom**. Role to tożsamości przyjmowane tymczasowo przez usługi i procesy. Działają na zasadzie najmniejszych uprawnień (*principle of least privilege*). Każda

tożsamość powinna posiadać wyłącznie uprawnienia niezbędne do swoich zadań. W zrealizowanym systemie role IAM posiadają m.in.: płaszczyzna sterowania klastrą, węzły robocze, sterownik EBS CSI.

AWS Secrets Manager uzupełnia IAM o scentralizowane przechowywanie sekretów aplikacyjnych jak haseł, kluczy i parametrów połączeń. Główną zaletą tego systemu jest szyfrowanie, kontrola dostępu i audyt przechowanych sekretów. Pozwala to nie trzymać haseł w kodzie źródłowym, a w pełni bezpiecznym środowisku.

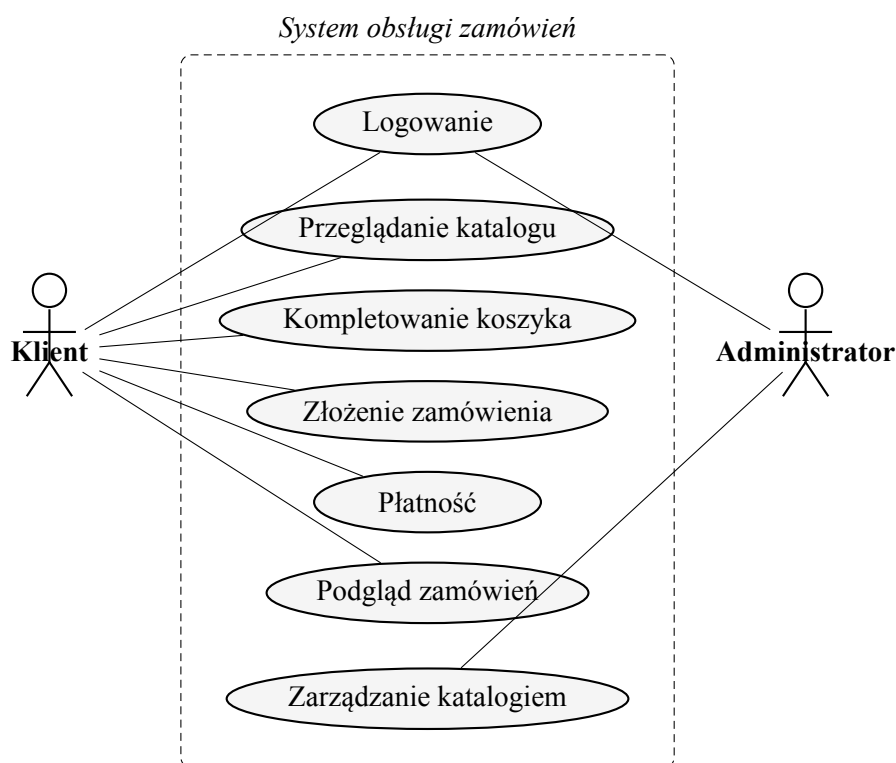
5. Projekt systemu obsługi zamówień

W tym rozdziale omówiony jest projekt zrealizowanego systemu obsługi zamówień. Uzasadnione zostanie wybór technologii oraz decyzji implementacyjnych. Decyzje projektowe odwołują się do podstaw teoretycznych omówionych w rozdziale 2.

5.1 Opis funkcjonalny systemu

System realizuje kompletny proces zakupowy w sklepie internetowym. Jego użytkownikami są **klienci**. Są to zarejestrowani użytkownicy przeglądający ofertę i składający zamówienia.

- rejestrację i logowanie użytkowników,
- przeglądanie katalogu produktów z filtrowaniem po kategorii, przedziale cenowym i dostępności,
- kompletowanie koszyka i składanie zamówień z adresem dostawy,
- automatyczną walidację zamówienia: weryfikację i rezerwację stanów magazynowych,
- obsługę procesu płatności za zamówienie (płatność symulowana),
- przeglądanie historii i szczegółów własnych zamówień, w tym pełnej historii zmian ich statusów,
- anulowanie nieprawidłowych zamówień.



Ilustracja 5.1: Diagram przypadków użycia systemu. Klient realizuje pełny proces zakupowy, administrator zarządza katalogiem produktów; logowanie jest wspólne dla obu ról.

Przebieg podstawowego scenariusza jest następujący. Klient kompletuje koszyk i składa zamówienie. System automatycznie weryfikuje dostępność zamówionych produktów i rezerwuje je. Po pomyślnej walidacji klient inicjuje płatność. Jej zakończenie przenosi zarezerwowane ilości produktów do puli opłaconych i finalizuje zamówienie. Niepowodzenie rezerwacji kończy proces automatycznym anulowaniem zamówienia. Przypadki użycia systemu wraz z rolami aktorów przedstawia rysunek 5.1.

5.2 Model domenowy

Domene systemu podzielono zgodnie z zasadą modelowania wokół kontekstów biznesowych (sekcja 2.2). Na trzy konteksty: *zamówienia*, *katalog produktów* oraz *płatności*.

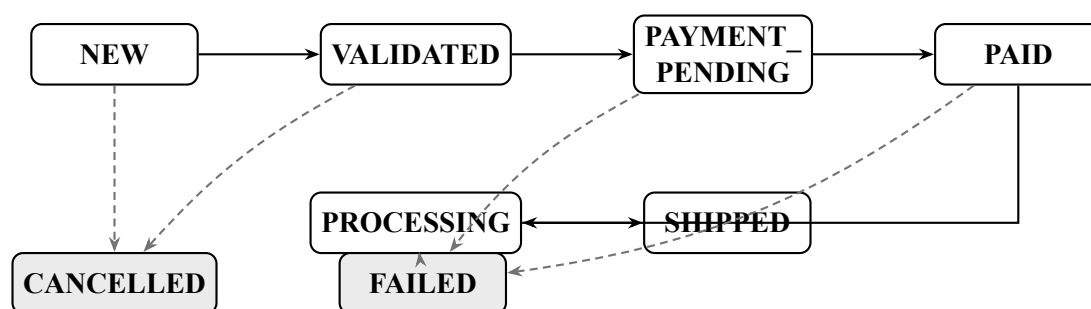
Zamówienie (Order) jest centralnym zdarzeniem systemu. Obejmuje indentyfikator klienta, listę pozycji (produkt, ilość, cena jednostkowa i łączna) kwotę całkowitą wraz z walutą, adres dostawy oraz bieżący status. Kwoty pieniężne modelowane są typem dziesiętnym o stałej precyzji. Eliminuje to błędy zaokrągleń w operacjach zmiennoprzecinkowych.

Cykl życia zamówienia zamodelowano jako maszynę stanów (wzorzec omówiony w sekcji 2.4). Osiem możliwych stanów zestawiono w tabeli tab:stany-zamowienia i zilustrowano na rysunku 5.2. Dozwolone przejścia tworzą ścieżkę główną: NEW → VALIDATED →

PAYMENT_PENDING → PAID → PROCESSING → SHIPPED. Przy czym z każdego stanu niekońcowego możliwe jest przejście do stanów CANCELLED lub FAILED. Stany SHIPPED, CANCELLED i FAILED są terminalne. Każda próba przejścia spoza tego zbioru jest odrzucana na poziomie logiki domenowej.

Tabela 5.1: Stany cyklu życia zamówienia

Stan	Rodzaj	Znaczenie
NEW	początkowy	zamówienie przyjęte, oczekuje na walidację
VALIDATED	pośredni	stany magazynowe zweryfikowane i zarezerwowane
PAYMENT_PENDING	pośredni	utworzono żądanie płatności
PAID	pośredni	płatność zakończona pomyślnie
PROCESSING	pośredni	zamówienie w realizacji
SHIPPED	terminalny	zamówienie wysłane
CANCELLED	terminalny	zamówienie anulowane (przez klienta lub system)
FAILED	terminalny	przetwarzanie zakończone błędem



Ilustracja 5.2: Maszyna stanów cyklu życia zamówienia. Linia ciągłą oznaczono ścieżkę główną, przerywaną — przejścia do stanów terminalnych CANCELLED i FAILED, osiągalne z każdego stanu nieterminalnego (dla czytelności pokazano reprezentatywne krawędzie). Stany terminalne wyróżniono szarym tłem.

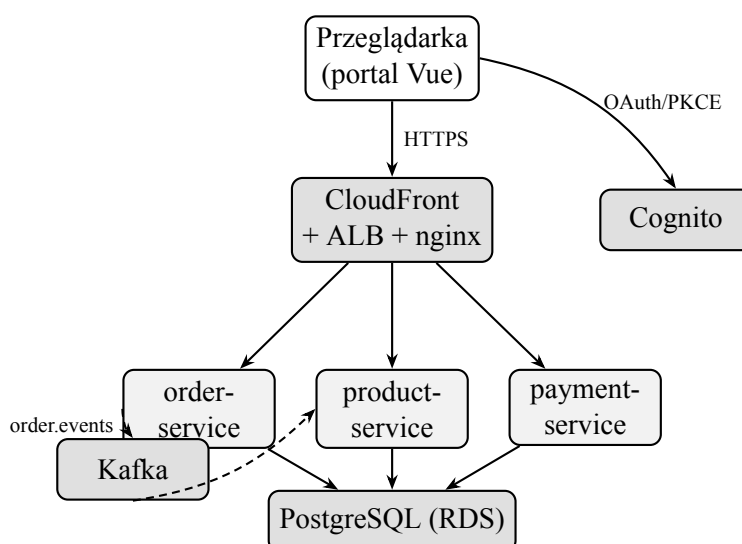
Produkt (Product) reprezentuje pozycję katalogu. Posiada on nazwę, kategorię, opis, cenę z walutą oraz opcjonalne zdjęcie. Istotnym elementem modelu jest rozbicie stanu magazynowego produktu na cztery pule ilościowe: `qty` (dostępne), `reserved_qty` (zarezerwowane w zamówieniach oczekujących na płatność), `paid_qty` (opłacone). Operacje biznesowe przesuwają ilości pomiędzy tymi pulami. Każde przesunięcie jest warunkowe i wymaga wystarczającej ilości w puli źródłowej. Chroni to przez sprzedaż ponad stan.

Płatność (Payment) wiąże zamówienie z procesem zapłaty. Posiada dwa stany: `PENDING` (utworzona, oczekuje na realizację) oraz `SUCCEEDED` (zakończona). Integracja z rzeczywistym operatorem płatności pozostaje po za zakresem pracy, więc realizacja płatności jest symulowana. Klient wytwarza żądanie o wytworzenie płatności. Serwis odpowiada adresem URL płatności

i ją finalizuje. Odpowiada to przepływowi stosowanemu przy integracjach zewnętrznych, co pozwala podmienić symulację na rzeczywistego operatora.

5.3 Architektura mikroserwisowa systemu

Każdemu kontekstowi domenowemu odpowiada osobny mikroservis: **order-service** (zamówienia), **product-service** (katalog i stany magazynowe) oraz **payment-service** (płatności). Interfejs użytkownika stanowi **portal obsługi zamówień**. Portal to aplikacja jednostronicowa działająca w przeglądarce. Architekturę systemu dopełnia broker komunikatów Apache Kafka, pośredniczący w asynchronicznej wymianie zdarzeń pomiędzy usługami. Relacyjna baza danych PostgreSQL. Usługa tożsamości Amazon Cognito. Warstwa brzegowa (CloudFront i load balancer), stanowiąca pojedynczy punkt wejścia do systemu. Całość przedstawia rysunek 5.3.



Ilustracja 5.3: Architektura systemu. Linia ciągła — komunikacja synchroniczna (HTTPS/REST), linia przerywana — asynchroniczna wymiana zdarzeń przez Apache Kafka. Cały ruch użytkownika przechodzi przez warstwę brzegową (CloudFront + ALB + nginx), a uwierzytelnianie realizuje Amazon Cognito. Komponenty walidacji JWT po stronie serwisów pominięto dla czytelności.

Ruch w systemie przebiega diemą ścieżkami. Żądania użytkownika są składane przez portla i trafiają poprzez warstwę przebogwą do właściwego serwisu. Dzieje się to za pomocą prefiksu ścieżki i są obsługiwane synchronicznie (sekcja 5.4). Porces walidacji nie wymaga udziału użytkownika, więc jest obsługowany asynchronicznie poprzez brokera wiadomości. Podział ten odpowiada zaleceniom przedstawionym w sekcji 2.3.

Świadomym odstępstwem od wzorca *database-per-service* jest wykorzystanie wspólnej instancji bazy danych: ze względów kosztowych usługi współdzielą jedną instancję PostgreSQL, zachowując jednak rozłączne zestawy tabel, których wyłącznym właścicielem pozostaje jedna usługa. Wyjątek stanowi proces finalizacji płatności, w którym payment-service aktualizuje

transakcyjnie również tabele zamówień i produktów — kompromis ten, jego konsekwencje oraz docelowe rozwiązanie omówiono w rozdziałach 6 i 9.

5.4 Projekt API REST/JSON oraz kontraktów zdarzeń

5.4.1 Interfejsy REST

Interfejsy usług zaprojektowano zgodnie z konwencjami REST: zasoby identyfikowane są ścieżkami, operacje metodami HTTP, a reprezentacją danych jest JSON. Zestawienie punktów końcowych przedstawia tabela 5.2. Wszystkie punkty końcowe — z wyjątkiem publicznego serwowania zdjęć produktów — wymagają uwierzytelnienia tokenem JWT (sekcja 5.8), a zasoby zamówień i płatności zwracane są wyłącznie ich właścicielowi, identyfikowanemu na podstawie tokenu. Listy stronicowane są parametrami `limit` i `offset`, a katalog produktów filtrowany parametrami zapytania (`category`, `minPrice`, `maxPrice`, `available`).

Tabela 5.2: Punkty końcowe REST mikroserwisów

Metoda	Ścieżka	Operacja
<i>order-service</i>		
POST	/api/orders	złożenie zamówienia (idempotentne)
GET	/api/orders	lista zamówień klienta
GET	/api/orders/{id}	szczegóły zamówienia
PATCH	/api/orders/{id}/status	zmiana statusu zamówienia
<i>product-service</i>		
GET	/api/products	lista produktów z filtrowaniem
GET	/api/products/{id}/image/{imageId}	pobranie zdjęcia (publiczne)
<i>payment-service</i>		
POST	/api/v1/payments	utworzenie żądania płatności
GET	/api/v1/payments	lista płatności klienta
GET	/api/v1/payments/{id}	szczegóły płatności
POST	/api/v1/payments/{id}/pay	realizacja płatności (symulowana)

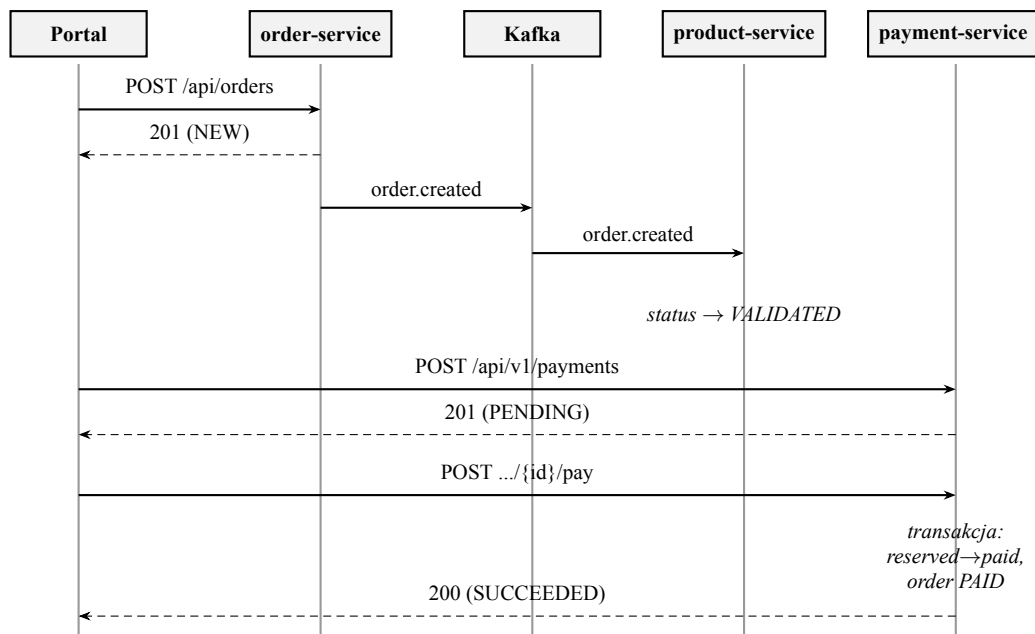
Operacja złożenia zamówienia zaprojektowana została jako idempotentna, według wzorca klucza idempotencji (sekcja 2.4). Klient przekazuje w nagłówku `Idempotency-Key` unikalny identyfikator operacji. Powtórzenie żądania z tym samym kluczem zwraca pierwotnie utworzone zamówienie zamiast tworzyć duplikat.

5.4.2 Kontrakty zdarzeń

Zdarzenia domenowe są przesyłane jako JSON. System order-service publikuje wiadomość na temat `order.events`. Product-service, jako konsument tego zdarzenia, podejmuje próbę atomowej rezerwacji wszystkich pozycji zamówienia. Zależnie od wyniku ustawia jego status na poprawny albo anulowany. Kwoty pieniężne są przesyłane jako znaki w celu uniknięcia utraty precyzji podczas deserializacji JSON. 5.1 Pełny przebieg procesu zakupowego przedstawia rysunek 5.4.

```
1 {
2   "event": "order.created",
3   "orderId": "550e8400-e29b-41d4-a716-446655440000",
4   "customerId": "c7f3a2e1-...",
5   "status": "NEW",
6   "totalAmount": "150.50",
7   "currency": "PLN",
8   "items": [
9     {
10      "productId": "9b2d...",
11      "productName": "Widget",
12      "quantity": 2,
13      "unitPrice": "75.25",
14      "totalPrice": "150.50"
15    }
16  ],
17   "occurredAt": "2026-06-04T10:30:00Z"
18 }
```

Listing 5.1: Struktura zdarzenia order.created



Ilustracja 5.4: Diagram sekwencji przepływu zakupowego. Po synchronicznym utworzeniu zamówienia jego walidacja przebiega asynchronicznie, według choreografii zdarzeń Kafki (order-service → product-service); płatność i jej finalizacja są ponownie operacjami synchronicznymi.

5.5 Wybór technologii implementacyjnych — Go i Vue 3

Na język implementacji mikroservisów wybrano **Go**. Język ten często określany jako język chmury [13]. Ma wiele zalet które idealnie sprawdzają się przy budowaniu mikroservisów. Go kompiluje się do pojedynczego, statycznie linkowanego pliku wykonywalnego. Pozwala to budować minimalne obrazy kontenerów pozbawione systemu operacyjnego (rozdział 6). Ma bardzo niskie zużycie pamięci i szybki start procesu. Posiada bardzo lekki model współbieżności (gorutyny). Jest to bardzo ważne przy serwerach obsługujących wiele żądań na raz. Go posiada też dojrzały ekosystem bibliotek. Spośród nich wykorzystano m.in. router HTTP `chi`, sterownik PostgreSQL `pgx`, klienta Kafki `franz-go` oraz bibliotekę `jwx` do weryfikacji tokenów JWT. Jedną z wielkich zalet Go jest też bardzo bogata biblioteka standardowa, która umożliwia nie używania ciężkich frameworków.

Warstwę frontendu zrealizowano we frameworku **Vue 3** z językiem **TypeScript**. Vue 3 oferuje reaktywny model danych i kompozycyjne API sprzyjające wielokrotnemu użyciu logiki. Statycznie typowany TypeScript pozwala wykryć znaczną ilość błędów w etapie kompilacji. Stan aplikacji (sesja użytkownika, koszyk) zarządzany jest biblioteką Pinia, a proces budowania narzędziem Vite. Szczegóły architektury portalu przedstawiono w rozdziale 7.

5.6 Projekt bazy danych i persystencji — PostgreSQL

Warstwę persystencji oparto na relacyjnej bazie danych **PostgreSQL**. PostgreSQL jest obecnie jedną z najlepiej rozwijanych baz danych. Bardzo dobra transakcyjność ACID była niezbędna przy warunkowych przesunięciach stanów magazynowych i finalizacji płatności. Dostępność PostgreSQL jako w pełni zarządzanej usługi Amazon RDS była kolejnym powodem dlaczego został wybrany ten silnik (sekcja 4.3.2).

```
1 CREATE TABLE orders (  
2     id                TEXT PRIMARY KEY,  
3     customer_id       TEXT      NOT NULL,  
4     status            TEXT      NOT NULL,  
5     items             JSONB      NOT NULL,  
6     total_amount      NUMERIC(20,4) NOT NULL,  
7     currency          CHAR(3)    NOT NULL,  
8     delivery_address  JSONB      NOT NULL,  
9     history           JSONB      NOT NULL,  
10    created_at        TIMESTAMPTZ NOT NULL,  
11    updated_at        TIMESTAMPTZ NOT NULL  
12 );
```

Listing 5.2: Schemat tabeli orders

Tabelę uzupełniają indeksy na kolumnach `customer_id` i `created_at`, wspierające najczęstsze zapytanie systemu. Tabela `idempotency_keys`, wiążąca klucze idempotencji z identyfikatorami utworzonych zamówień.

```
1 CREATE TABLE idempotency_keys (  
2     key                TEXT PRIMARY KEY,  
3     order_id          TEXT      NOT NULL,  
4     created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()  
5 );
```

Listing 5.3: Schemat tabeli idempotency keys

Serwis produktów przechowuje katalog w tabeli `products`. Stan magazynowy zamodelowano czterema kolumnami ilościowymi (`qty`, `reserved_qty`, `paid_qty`, `shipped_qty`) z ograniczeniami `CHECK` wykluczającymi wartości ujemne. Dzięki czemu niezmiennik nieujemności stanów magazynowych egzekwowany jest również na poziomie bazy.

```
1 CREATE TABLE products (  
2     id                TEXT PRIMARY KEY,  
3     name              TEXT      NOT NULL,
```

```

4      category      TEXT          NOT NULL,
5      description   TEXT          NOT NULL,
6      price         NUMERIC(20,4) NOT NULL,
7      currency      CHAR(3)       NOT NULL,
8      qty           INTEGER       NOT NULL CHECK (qty >= 0),
9      reserved_qty  INTEGER       NOT NULL DEFAULT 0 CHECK (reserved_qty
10     >= 0),
11     paid_qty      INTEGER       NOT NULL DEFAULT 0 CHECK (paid_qty >=
12     0),
13     shipped_qty   INTEGER       NOT NULL DEFAULT 0 CHECK (shipped_qty
14     >= 0),
15     created_at    TIMESTAMPTZ   NOT NULL,
16     updated_at    TIMESTAMPTZ   NOT NULL
17 );

```

Listing 5.4: Schemat tabeli products

Zdjęcia produktów przechowywane są w tabeli `product_images` w kolumnie binarnej `BYTEA`, wraz z typem `MIME` i rozmiarem.

```

1 CREATE TABLE IF NOT EXISTS product_images (
2     product_id    TEXT PRIMARY KEY REFERENCES products (id) ON DELETE
3     CASCADE,
4     id            TEXT          NOT NULL,
5     content_type   TEXT          NOT NULL,
6     file_name      TEXT,
7     byte_size      BIGINT       NOT NULL CHECK (byte_size >= 0),
8     data           BYTEA        NOT NULL,
9     created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW()
10 );

```

Listing 5.5: Schemat tabeli images

Serwis płatności utrzymuje tabelę `payments`, wiążącą płatność z zamówieniem i klientem oraz rejestrującą znacznik czasu finalizacji.

```

1 CREATE TABLE IF NOT EXISTS payments (
2     id            TEXT PRIMARY KEY,
3     order_id      TEXT          NOT NULL REFERENCES orders (id),
4     customer_id   TEXT          NOT NULL,
5     status        TEXT          NOT NULL,
6     created_at    TIMESTAMPTZ   NOT NULL,
7     updated_at    TIMESTAMPTZ   NOT NULL,
8     completed_at  TIMESTAMPTZ

```

Listing 5.6: Schemat tabeli payments

5.7 Asynchroniczna wymiana zdarzeń — Apache Kafka

Rolę brokera komunikatów pełni **Apache Kafka**. Jest to rozproszony dziennik zdarzeń (*distributed log*), w którym komunikaty zapisywane są trwale, w kolejności, do nazwanych tematów (*topics*) dzielonych na partycje [8]. Model konsumpcji oparty jest na grupach konsumentów. Każda grupa utrzymuje własne przesunięcie (*offset*) w temacie, dzięki czemu wielu niezależnych odbiorców może przetwarzać ten sam strumień zdarzeń, a konsument po awarii wznowia przetwarzanie od ostatniej zatwierdzonej pozycji. Ta właściwość odróżnia Kafkę od klasycznych kolejek komunikatów i czyni ją naturalnym nośnikiem zdarzeń domenowych.

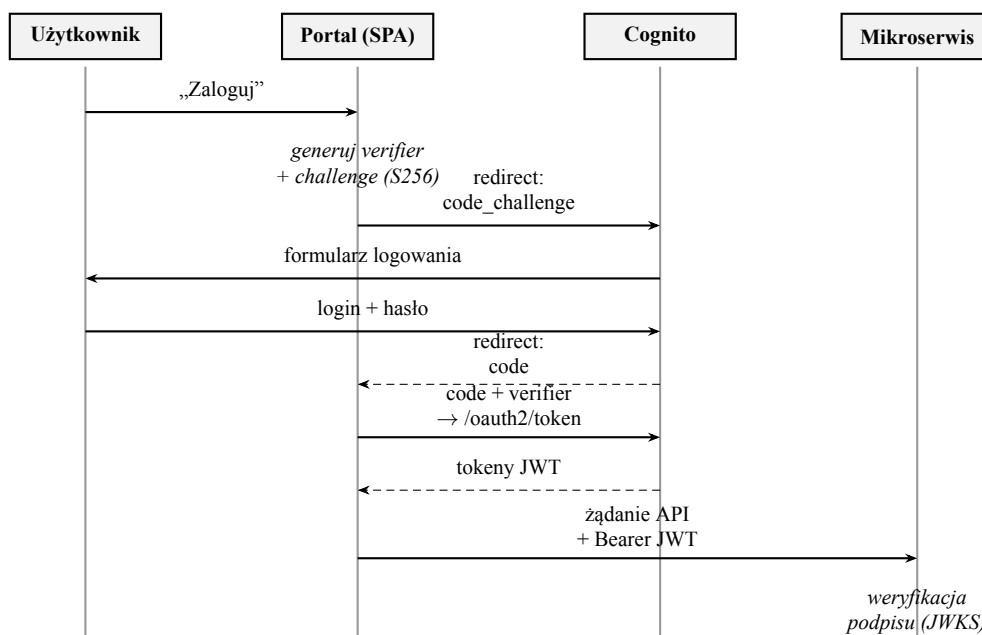
W systemie zdefiniowano dwa tematy. `order.events`, do którego `order-service` publikuje zdarzenia cyklu życia zamówień. `product.events`, do którego `product-service` publikuje wyniki walidacji. `Product-service` konsumuje temat `order.events` w ramach grupy `product-service`. Kafka gwarantuje dostarczenie *co najmniej raz*, zatem konsumenci zaprojektowani są idempotentnie. Powtórne przetworzenie zdarzenia `order.created` nie podwaja rezerwacji, ponieważ operacja rezerwacji dla danego zamówienia jest skuteczna tylko przy pierwszym wykonaniu. Broker uruchamiany jest w klastrze Kubernetes w trybie KRaft (bez komponentu ZooKeeper), w konfiguracji jednowęzłowej. Wystarcza to dla środowiska demonstracyjnego, której ograniczenia omówiono w rozdziale 9.

5.8 Projekt uwierzytelniania i autoryzacji

Uwierzytelnianie użytkowników oparto na protokole **OAuth 2.0** [14] z warstwą tożsamości OpenID Connect, z Amazon Cognito w roli serwera autoryzacji (sekcja 4.3.4). Ponieważ klientem jest aplikacja jednostronicowa, zastosowano przepływ *authorization code* z rozszerzeniem **PKCE** (*Proof Key for Code Exchange*) [15]. Portal generuje przy każdym logowaniu jednorazowy weryfikator, którego skrót dołącza do żądania autoryzacji, a oryginał przedstawia przy wymianie kodu na tokeny. Uniemożliwia wykorzystanie przechwyconego kodu autoryzacyjnego przez stronę trzecią.

Przebieg logowania jest następujący. Portal przekierowuje użytkownika do hostowanego interfejsu logowania Cognito. Po poprawnym uwierzytelnieniu Cognito zwraca kod autoryzacyjny na zarejestrowany adres zwrotny portalu, który wymienia go wraz z weryfikatorem PKCE na tokeny JWT. Hasła użytkowników nigdy nie przepływają przez aplikację. Uzyskany token

dołączany jest do każdego żądania API w nagłówku `Authorization`.



Ilustracja 5.5: Przebieg uwierzytelniania OAuth 2.0 *authorization code* z rozszerzeniem PKCE. Portal generuje weryfikator i jego skrót; po zalogowaniu wymienia kod autoryzacyjny — wraz z weryfikatorem — na tokeny JWT. Każdy mikroserwis weryfikuje podpis tokenu samodzielnie, na podstawie kluczy publicznych Cognito (JWKS).

Autoryzację zaprojektowano w modelu zdecentralizowanym. Każdy mikroserwis samodzielnie weryfikuje podpis i zawartość tokenu na podstawie publicznych kluczy Cognito (zbioru JWKS), bez odwołań do centralnej usługi sesji. Dzięki czemu uwierzytelnienie żądania nie wprowadza dodatkowej zależności sieciowej ani pojedynczego punktu awarii. Tożsamość klienta odczytywana z tokenu wyznacza zakres dostępu do danych (klient widzi wyłącznie własne zamówienia i płatności). Szczegóły weryfikacji tokenów po stronie usług przedstawiono w rozdziale 6, a implementację przebiegu PKCE w portalu w rozdziale 7.

6. Implementacja mikroservisów

Rozdział ten przedstawia implementację trzech mikroservisów systemu w języku Go. Omówiono wspólną strukturę i wzorce dzielone przez wszystkie usługi, a następnie logikę biznesową każdej z nich, warstwę zdarzeń, mechanizm uwierzytelniania, obserwowalność, konteneryzację oraz testy. Prezentowane listingi pochodzą bezpośrednio z kodu źródłowego systemu.

6.1 Struktura projektu i wspólne wzorce servisów

Wszystkie trzy serwisy zbudowano według tego samego szablonu, opartego na architekturze heksagonalnej (*ports and adapters*). Logika domenowa stanowi rdzeń niezależny od technologii. Komunikacja ze światem zewnętrznym przebiega przez HTTP. Baza danych, broker komunikatów odbywa się poprzez wymienne adaptery, sprzęgnięte z rdzeniem wyłącznie interfejsami. Strukturę katalogów serwisu zamówień przedstawia listing 6.1.

```
1 order-service/
2 |-- main.go           # punkt śwejcia, łżzowanie żśzalenoci
3 |-- Dockerfile
4 `-- internal/
5     |-- auth/         # weryfikacja JWT (Cognito)
6     |-- config/       # konfiguracja ze zmiennych środowiskowych
7     |-- db/           # pula łańpocze PostgreSQL, migracje
8     |-- metrics/      # definicje metryk Prometheus
9     |-- producer/     # producent ŗzdarze Kafka
10    |-- server/       # serwery HTTP: API i techniczny
11    `-- order/        # kontekst domenowy
12        |-- domain/   # encje, maszyna stanów, niezmienniki
13        |-- service/  # przypadki życia (logika aplikacyjna)
14        |-- repository/ # adapter PostgreSQL
15        |-- events/   # definicje ŗzdarze domenowych
16        `-- api/     # handlersy HTTP, DTO, walidacja śwejcia
```

Listing 6.1: Struktura katalogów serwisu zamówień

Zależności składane są jawnie w funkcji `main`, bezwstrzykiwania zależności. Granice pomiędzy warstwami wyznaczają małe interfejsy definiowane po stronie konsumenta, co jest idiomatycznym wzorcem języka Go. Tworzy to pakiety bez zewnętrznych zależności co pozwala testować logikę biznesową w odpowiedniej izolacji.

Każdy serwis uruchamia dwa niezależne serwery HTTP na oddzielnych portach. Serwer **API** obsługuje ruch biznesowy. Serwer **techniczny** udostępnia punkty końcowe eksploatacyjne: `/ping` (kontrola zdrowia), `/metrics` (metryki Prometheus), `/version` oraz `/debug/pprof` (profilowanie procesu). Rozdzielenie portów gwarantuje, że dane eksploatacyjne nigdy nie są osiągalne ścieżką publiczną, nawet przy błędnej konfiguracji routingu.

Konfiguracja serwisów realizuje wzorec konfiguracji zewnętrznej (sekcja 2.4). Wszystkie parametry wczytywane są ze zmiennych środowiskowych do struktur Go.

6.2 Serwis zamówień - logika biznesowa, maszyna stanów i idempotencja

Serwis zamówień realizuje utworzenie zamówienia oraz pobranie i listowanie zamówień klienta. Tworząc zamówienie, warstwa domenowa egzekwuje niezmienniki: niepustą listę pozycji, dodatnie ilości, poprawny kod waluty oraz zgodność kwoty całkowitej z sumą pozycji.

Centralnym elementem domeny jest maszyna stanów zamówienia. Dozwolone przejścia zakodowano jako strukturę danych. Mapę stanów osiągalnych z każdego stanu, dzięki czemu cykl życia zamówienia opisany jest w jednym miejscu kodu i bezpośrednio odpowiada diagramowi z projektu (sekcja 5.2). Implementację przedstawia listing 6.2. Stany terminalne nie posiadają wpisu w mapie, więc żadne przejście z nich nie jest możliwe.

```
1 // allowedTransitions defines the order lifecycle.
2 //
3 // NEW -> VALIDATED -> PAYMENT_PENDING -> PAID -> PROCESSING ->
  SHIPPED
4 // plus CANCELLED / FAILED reachable from any non-terminal state.
5 var allowedTransitions = map[OrderStatus]map[OrderStatus]struct{
6     OrderStatusNew:           {OrderStatusValidated: {}},
7     OrderStatusCancelled:     {}, OrderStatusFailed: {}},
8     OrderStatusValidated:     {OrderStatusPaymentPending: {}},
9     OrderStatusCancelled:     {}, OrderStatusFailed: {}},
10    OrderStatusPaymentPending: {OrderStatusPaid: {}},
    OrderStatusCancelled:     {}, OrderStatusFailed: {}},
    OrderStatusPaid:           {OrderStatusProcessing: {}},
    OrderStatusCancelled:     {}, OrderStatusFailed: {}},
    OrderStatusProcessing:     {OrderStatusShipped: {}},
```

```

    OrderStatusCancelled: {}, OrderStatusFailed: {}},
11 }
12
13 func CanTransition(from, to OrderStatus) bool {
14     if from == to {
15         return false
16     }
17     next, ok := allowedTransitions[from]
18     if !ok {
19         return false
20     }
21     _, ok = next[to]
22     return ok
23 }

```

Listing 6.2: Implementacja maszyny stanów zamówienia

Każde wykonane przejście dopisywane jest do historii zamówienia (stan poprzedni, nowy, znacznik czasu, opcjonalny powód), utrwalanej w kolumnie `JSONB`. Daje to pełny ślad audytowy cyklu życia każdego zamówienia.

Idempotencję operacji utworzenia zamówienia zrealizowano na poziomie bazy danych, w sposób odporny na wyścigi współbieżnych żądań (listing 6.3). Klucz z nagłówka `Idempotency-Key` zapisywany jest instrukcją `INSERT` z klauzulą `ON CONFLICT`. Jeśli klucz został już zajęty, zapytanie zamiast zakończyć się błędem zwraca identyfikator zamówienia utworzonego pierwotnie, a serwis odpowiada klientowi tym właśnie zamówieniem. Rozstrzygnięcie kolizji następuje zatem atomowo w jednym zapytaniu, bez okna czasowego pomiędzy sprawdzeniem a zapisem klucza.

```

1 func (r *Postgres) SaveIdempotencyKey(ctx context.Context, key,
   orderID string) (string, error) {
2     var stored string
3     err := r.pool.QueryRow(ctx, `
4         INSERT INTO idempotency_keys (key, order_id) VALUES ($1, $2)
5         ON CONFLICT (key) DO UPDATE SET key = idempotency_keys.key
6         RETURNING order_id
7     `, key, orderID).Scan(&stored)
8     if err != nil {
9         return "", fmt.Errorf("save idempotency key: %w", err)
10    }
11    if stored != orderID {
12        return stored, domain.ErrIdempotencyExists
13    }

```

```
14     return "", nil
15 }
```

Listing 6.3: Atomowe zajęcie klucza idempotencji

Po utrwaleniu zamówienia serwis publikuje zdarzenie `order.created`.

6.3 Serwis produktów - katalog i rezerwacja stanów magazynowych

Serwis produktów pełni dwie role. Synchronicznie obsługuje katalog (listowanie z filtrowaniem po kategorii, przedziale cenowym i dostępności, obsługę zdjęć przechowywanych w kolumnie `BYTEA`). Oraz asynchronicznie uczestniczy w walidacji zamówień jako konsument zdarzeń.

Rezerwacja stanów magazynowych realizowana jest operacją wsadową obejmującą wszystkie pozycje zamówienia w jednej transakcji bazodanowej. Rezerwacja jest więc niepodzielna. Nie istnieje stan, w którym część pozycji zamówienia została zarezerwowana, a część nie. Samo przesunięcie ilości pomiędzy pulami wykonywane jest warunkową instrukcją `UPDATE`. Warunek (`qty >= $2`) gwarantuje poprawność również przy współbieżnej rezerwacji tego samego produktu przez dwa zamówienia. Przegrywające zapytanie nie zmodyfikuje żadnego wiersza, co serwis interpretuje jako niewystarczający stan magazynowy.

Po otrzymaniu zdarzenia `order.created` konsument dekoduje je, waliduje i podejmuje próbę rezerwacji. W zależności od wyniku publikuje `order.validated` albo `order.reservation_failed`.

Rozwiązaniem przejściowym, jest sposób domknięcia pętli walidacji. Docelowo `order-service` powinien konsumować temat `product.events` i sam zmieniać status zamówienia. W bieżącej wersji systemu konsument ten nie istnieje. `Product-service` aktualizuje więc status zamówienia (`VALIDATED` lub `CANCELLED`) bezpośrednio w tabeli zamówień, równolegle z publikacją zdarzenia. Kompromis ten narusza zasadę wyłącznej własności danych (sekcja 2.2) i został świadomie przyjęty jako etap pośredni. Jego usunięcie wskazano jako kierunek rozwoju w rozdziale 10.

6.4 Serwis płatności - transakcyjna obsługa procesu płatności

Serwis płatności obsługuje utworzenie żądania płatności dla zamówienia (zwracając klientowi adres URL płatności) oraz jej finalizację. Finalizacja jest najbardziej krytyczną operacją systemu. Musi jednocześnie i niepodzielnie przenieść zarezerwowane ilości wszystkich po-

zycji zamówienia do puli opłaconych, oznaczyć zamówienie jako `PAID` oraz płatność jako `SUCCEEDED`. Zrealizowano ją jako pojedynczą transakcję bazodanową z blokadami wierszy.

Transakcja rozpoczyna się pobraniem płatności i zamówienia z klauzulą `FOR UPDATE`, co wyklucza równoległą finalizację tej samej płatności. Następnie weryfikowane są warunki: należność płatności i zamówienia do uwierzytelnionego klienta, poprawność pozycji oraz to, czy zamówienie znajduje się w stanie dopuszczającym zapłatę. Przy czym ponowna finalizacja płatności już zakończonej nie jest błędem, lecz zwraca jej bieżący stan (operacja jest idempotentna). Dla każdej pozycji zamówienia wykonywane jest warunkowe przesunięcie ilości (listing 6.4). Niepowodzenie któregośkolwiek przesunięcia wycofuje całą transakcję.

```
1 tag, err := tx.Exec(ctx, `
2     UPDATE products
3     SET reserved_qty = reserved_qty - $2,
4         paid_qty = paid_qty + $2
5     WHERE id = $1 AND reserved_qty >= $2
6 `, item.ProductID, item.Quantity)
```

Listing 6.4: Warunkowe przesunięcie ilości w transakcji płatności

Transakcja ta obejmuje tabele trzech kontekstów domenowych (płatności, zamówień i produktów), co stanowi opisany w sekcji 5.3 świadomy kompromis względem wzorca *database-per-service*. Współdzielenie jednej instancji baz gwarantujący silną spójność najbardziej wrażliwej operacji systemu bez implementacji rozproszonej sagi.

6.5 Publikacja i konsumpcja zdarzeń Kafka

Warstwę zdarzeń zrealizowano biblioteką `franz-go`. Producent serializuje zdarzenia do JSON i publikuje je do skonfigurowanego tematu w trybie nieblokującym. Publikacja nie wydłuża obsługi żądania HTTP z buforowaniem

Konsument w `product-service` działa w ramach grupy konsumentów `product-service`, dzięki czemu pozycja odczytu tematu `order.events` jest trwała. Po restarcie serwis wznowia przetwarzanie od ostatniego zatwierdzonego przesunięcia, nie gubiąc zdarzeń opublikowanych w czasie jego niedostępności.

6.6 Walidacja tokenów JWT i integracja z Cognito

Uwierzytelnianie żądań zrealizowano jako middleware HTTP, włączany przed handlersy wszystkich chronionych punktów końcowych. Middleware wydobywa token z nagłówka

Authorization, przekazuje go do weryfikatora, a po pomyślnej weryfikacji umieszcza tożsamość klienta w kontekście żądania. Skąd odczytują ją handlers, zawężając operacje do zasobów uwierzytelnionego użytkownika.

Produkcyjny weryfikator (listing 6.5) opiera się na bibliotece `jwt`. Klucze publiczne puli użytkowników pobierane są ze standardowego adresu JWKS i buforowane lokalnie z okresowym odświeżaniem (od 15 minut do 24 godzin).

```
1 func (v *CognitoVerifier) Verify(_ context.Context, raw string) (jwt.  
    Token, error) {  
2     tok, err := jwt.Parse(  
3         []byte(raw),  
4         jwt.WithKeySet(v.keySet),  
5         jwt.WithValidate(true),  
6         jwt.WithIssuer(v.issuer),  
7         jwt.WithAcceptableSkew(30*time.Second),  
8     )  
9     if err != nil {  
10        return nil, fmt.Errorf("token verify: %w", err)  
11    }  
12  
13    var tokenUse string  
14    if err := tok.Get("token_use", &tokenUse); err != nil {  
15        return nil, fmt.Errorf("token_use claim missing: %w", err)  
16    }  
17    if _, ok := v.allowedTokenUse[tokenUse]; !ok {  
18        return nil, fmt.Errorf("token_use %q not allowed", tokenUse)  
19    }  
20    ...  
21 }
```

Listing 6.5: Weryfikacja tokenu JWT wobec kluczy Cognito

Na potrzeby pracy lokalnej przewidziano tryb deweloperski (`COGNITO_ENABLED=false`), w którym tożsamość klienta przekazywana jest jawnie w nagłówku żądania. Tryb ten nie jest dostępny w konfiguracji produkcyjnej.

6.7 Obserwowalność serwisów

Każdy serwis publikuje metryki Prometheus w przestrzeni nazw odpowiadającej usłudze (prefiksy `order_`, `product_`, `payment_`). Warstwa HTTP instrumentowana jest middle-

ware'em zliczającym żądania, mierzącym histogram czasów obsługi oraz liczbę żądań w toku. Obok metryk technicznych serwisy publikują metryki domenowe, które pozwalają obserwować system w kategoriach biznesowych, a nie wyłącznie infrastrukturalnych.

Logowanie zrealizowano pakietem standardowym `slog` z formatem JSON i konfigurowalnym poziomem. Serwer techniczny każdej usługi udostępnia ponadto kontrolę zdrowia `/ping`, informację o wersji `/version` oraz profilowanie `pprof`, pozwalające diagnozować zużycie procesora i pamięci działającego procesu.

6.8 Konteneryzacja serwisów

Obrazy kontenerów budowane są wieloetapowo (listing 6.6). Etap budowania, oparty na obrazie `golang:alpine`, kompiluje statycznie linkowany plik wykonywalny. Osobne kopiowanie plików `go.mod` i `go.sum` oraz montowanie pamięci podręczne modułów i kompilatora pozwalają przy niezmienionych zależnościach pominąć ich ponowne pobieranie. Obraz docelowy to `distroless/static`. Czyli obraz pozbawiony powłoki, menedżera pakietów i jakichkolwiek narzędzi systemowych. Zawierający wyłącznie skompilowany plik binarny i certyfikaty TLS. Uruchamiany jest jako użytkownik nieuprzywilejowany (`nonroot`). Minimalizuje to zarówno rozmiar obrazu, jak i powierzchnię ataku.

```
1 FROM golang:1.26.3-alpine AS build
2 WORKDIR /src
3
4 COPY go.mod go.sum ./
5 RUN --mount=type=cache,target=/go/pkg/mod \
6     go mod download
7
8 COPY . .
9 RUN --mount=type=cache,target=/go/pkg/mod \
10     --mount=type=cache,target=/root/.cache/go-build \
11     CGO_ENABLED=0 GOOS=linux go build \
12     -trimpath -ldflags="-s -w" \
13     -o /out/order-service .
14
15 FROM gcr.io/distroless/static-debian12:nonroot
16 COPY --from=build /out/order-service /order-service
17 EXPOSE 9002 9090
18 USER nonroot
19 ENTRYPOINT ["/order-service"]
```

Listing 6.6: Wieloetapowy Dockerfile serwisu zamówień

6.9 Testy jednostkowe i integracyjne

Testy serwisów napisano w standardowym frameworku testowym Go. W konwencji testów tabelarycznych. Pokrywają one dwa poziomy. Poziom domenowy weryfikuje logikę biznesową w pełnej izolacji.

Testy uruchamiane są z włączonym detektorem wyścigów (`go test -race`) oraz losową kolejnością wykonywania (`-shuffle=on`). Pozwala to wykrywać odpowiednio błędy współbieżności i ukryte zależności pomiędzy testami. Te same polecenia wykonywane są automatycznie przy każdej zmianie kodu w potoku ciągłej integracji. Testy obciążeniowe systemu wdrożonego w środowisku docelowym omówiono odrębnie w rozdziale 9.

7. Implementacja portalu obsługi zamówień

Interfejs użytkownika systemu stanowi portal obsługi zamówień. Aplikacja jednostronowa (SPA) działająca w przeglądarce, realizująca pełny przepływ zakupowy. Od przeglądania katalogu, przez koszyk i złożenie zamówienia, po płatność i wgląd w historię zamówień. Rozdział omawia architekturę aplikacji, jej widoki, warstwę komunikacji z interfejsami REST mikroserwisów, implementację logowania w przepływie OAuth 2.0 z PKCE oraz testy.

7.1 Architektura aplikacji frontendowej

Aplikację zbudowano we frameworku Vue 3 z językiem TypeScript w trybie ścisłym (`strict`), z użyciem kompozycyjnego API. Kod podzielony jest na warstwy o jasno określonych odpowiedzialnościach. Katalog `views` zawiera widoki powiązane z trasami routera. Katalog `components` zawiera współdzielone komponenty powłoki aplikacji, Ścieżka `stores` zawiera magazyny stanu Pinia, a `services` warstwę dostępu do API. Na koniec `composables` ma funkcje kompozycyjne wielokrotnego użytku. Warstwę prezentacji ostyleowano biblioteką Tailwind CSS z paletą i typografią wzorowaną na Material Design 3. Nawigację obsługuje Vue Router z dziesięcioma trasami.

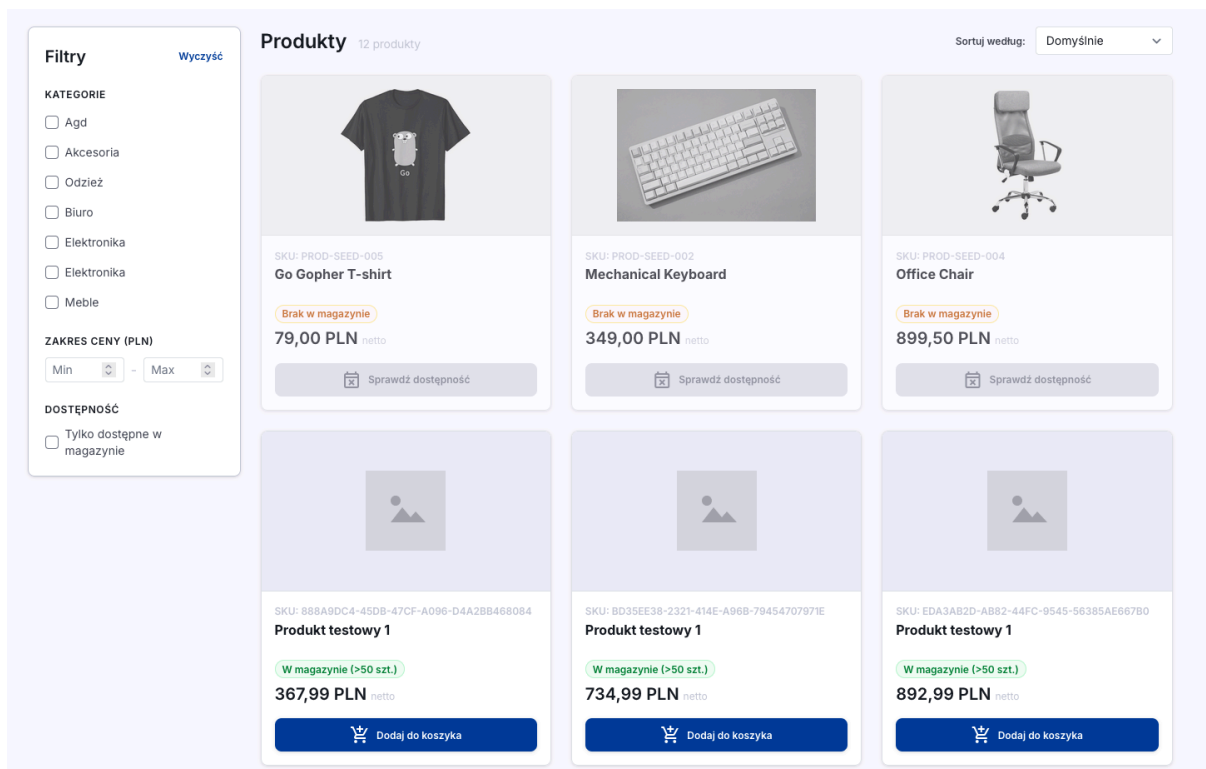
Globalny stan aplikacji ograniczono do dwóch magazynów Pinia. Magazyn `auth` przechowuje dane zalogowanego użytkownika. Magazyn `cart` przechowuje pozycje koszyka jako pary identyfikator-ilość i synchronizuje je reaktywnie z pamięcią `localStorage` pod kluczem `oms.cart`. Dzięki czemu koszyk przetrwa odświeżenie strony i ponowne otwarcie przeglądarki. Koszyk celowo nie przechowuje cen ani nazw produktów. Przy każdym wyświetleniu funkcja kompozycyjna `useCartDetails` łączy pozycje koszyka z aktualnymi danymi katalogu, więc prezentowane ceny i dostępność są zawsze bieżące.

7.2 Główne widoki i przepływy użytkownika

Przepływ zakupowy przebiega przez kolejne widoki aplikacji.

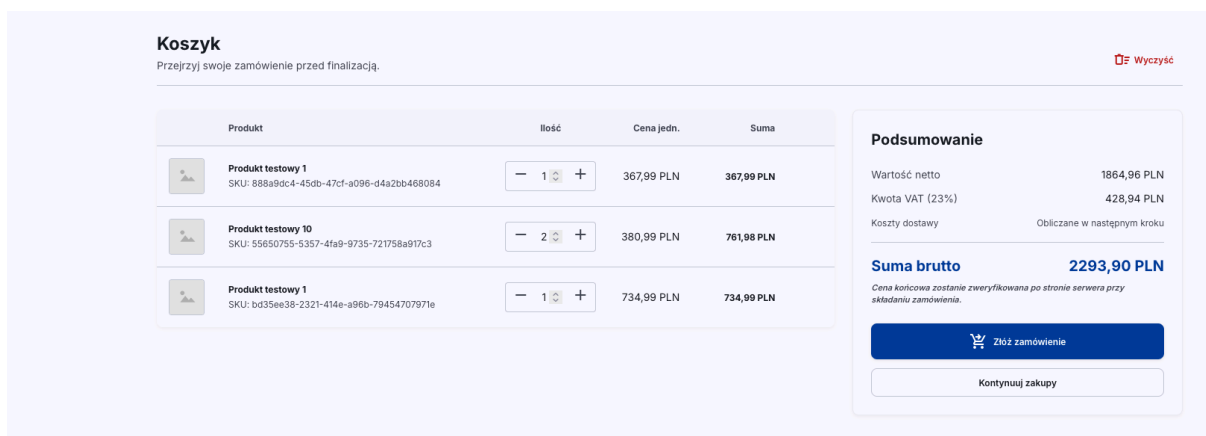
Katalog (`CatalogView`) prezentuje stronicowaną listę produktów z filtrowaniem po ka-

tegorii, przedziale cenowym i dostępności. Przy czym produkty o wyczerpanym stanie magazynowym mają zablokowaną możliwość dodania do koszyka.



Ilustracja 7.1: Widok katalogu produktów zrealizowanego serwisu.

Koszyk (CartView) umożliwia edycję ilości i usuwanie pozycji,



Ilustracja 7.2: Widok koszyka zrealizowanego serwisu.

Finalizacja (CheckoutView) zbiera adres dostawy i składa zamówienie.

[← Wróć do koszyka](#)

Dane zamówienia

Podaj adres dostawy, aby sfinalizować zamówienie.

Adres dostawy

Ulica i numer

ul.Przemysłowa

Kod pocztowy

03-555

Miasto

Warszawa

Kraj

Polska

Podsumowanie

Wartość netto

1864,96 PLN

Kwota VAT (23%)

428,94 PLN

Suma brutto

2293,90 PLN

Cena końcowa zostanie zweryfikowana po stronie serwera przy składaniu zamówienia.

✓ Złóż zamówienie

Pozycje zamówienia

Produkt testowy 1 1 × 367,99 PLN	367,99 PLN
Produkt testowy 10 2 × 380,99 PLN	761,98 PLN
Produkt testowy 1 1 × 734,99 PLN	734,99 PLN

Ilustracja 7.3: Widok finalizacji zamówienia zrealizowanego serwisu.

Po jego utworzeniu użytkownik przechodzi do **szczegółów zamówienia** (`OrderDetailsView`). Prezentujących pozycje, adres, bieżący status wraz z wizualizacją postępu w cyklu życia.

[← Wróć do listy](#)

Zamówienie #f63dbf92-9cd7-41a6-b61e-059f64136aac

Data złożenia: 8 cze 2026, 15:55

Opłać zamówienie

Status Zamówienia

NEW

VALIDATED

PAYMENT_PENDING

PAID

PROCESSING

SHIPPED

Zamówione Produkty

Lp.	Produkt	Ilość	Cena Netto	Wartość Netto
1	Produkt testowy 1 SKU: 888a9dc4-45db-47cf-a096-d4a2bb468084	1 szt.	367,99 PLN	367,99 PLN
2	Produkt testowy 10 SKU: 55650755-5357-4fa9-9735-721758a917c3	2 szt.	380,99 PLN	761,98 PLN
3	Produkt testowy 1 SKU: bd35ee38-2321-414e-a96b-79454707971e	1 szt.	734,99 PLN	734,99 PLN

Podsumowanie

Wartość produktów netto:

1864,96 PLN

Koszty dostawy netto:

0,00 PLN

Razem netto:

1864,96 PLN

VAT (23%):

428,94 PLN

Do zapłaty:

2293,90 PLN

Dane Dostawy

ADRES DOSTAWY

ul.Przemysłowa

03-555 Warszawa Polska

Ilustracja 7.4: Widok szczegółów zamówienia zrealizowanego serwisu.

Przycisk rozpoczęcia płatności prowadzi do **widoku płatności** (`PaymentView`) z symulowanym formularzem zapłaty.

Wybierz metodę płatności

Zamówienie nr: f63dbf92-9cd7-41a6-b61e-059f64136aac

Karta płatnicza

BLIK

Przelew bankowy

Dane karty

Numer karty

**** * 4242

Data ważności

12/25

CVC/CVV

Imię i nazwisko posiadacza

Jan Kowalski

Podsumowanie

Do zapłaty **1864,96 PLN**

Kwota brutto zweryfikowana po stronie serwera na podstawie zamówienia.

Zapłać 1864,96 PLN

Transakcja szyfrowana 256-bit SSL

Ilustracja 7.5: Widok płatności zrealizowanego serwisu.

Historię zamówień prezentuje OrdersView,

Historia Zamówień

Przeglądaj i zarządzaj swoimi dotychczasowymi zamówieniami.

Eksportuj Listę

Szukaj po numerze

Np. ZAM-2023-10-001

Status

Wszystkie statusy

Zakres dat

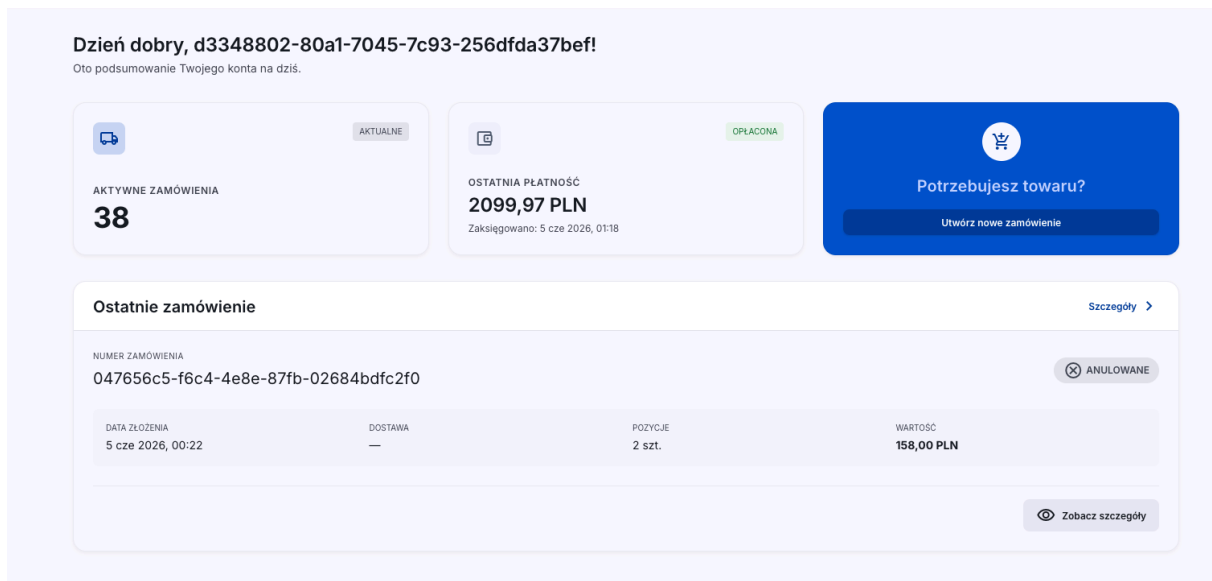
Ostatnie 30 dni

Wyczyść

Numer Zamówienia	Data	Kwota	Status	Akcje
4801d3f5-9df6-40b1-a55a-20fcf7cd9d46	3 cze 2026, 16:30	1628,99 PLN	OCZEKUJĄCE	Szczegóły
b9d7087f-faa5-4897-8cbb-765d872f7bfc	3 cze 2026, 17:37	1848,00 PLN	OCZEKUJĄCE	Szczegóły
2a9aa606-d8ea-4b8f-984e-eaf0855dc191	3 cze 2026, 17:55	1578,00 PLN	OCZEKUJĄCE	Szczegóły
0629fc12-9c2f-498f-8337-17b4b7d13ff8	3 cze 2026, 19:13	2056,99 PLN	OCZEKUJĄCE	Szczegóły
d223bc72-6ad3-4bb8-a304-0cac107200ba	3 cze 2026, 19:23	1578,00 PLN	OPLACONE	Szczegóły
22686aef-7b6e-4979-a827-c6a6659665aa	3 cze 2026, 21:14	428,00 PLN	OPLACONE	Szczegóły
4cc382e8-b275-4d40-ba63-be554984ca74	3 cze 2026, 21:43	395,00 PLN	OCZEKUJĄCE	Szczegóły
ae2c9693-4394-4e9a-aa82-9f8c93d3ec76	3 cze 2026, 21:45	79,00 PLN	OCZEKUJĄCE	Szczegóły
f6376681-2e2d-4e52-8b92-5911b8caa589	3 cze 2026, 21:52	428,00 PLN	OPLACONE	Szczegóły
338a2cbd-d624-4de3-a5ca-1e62f16eb07f	5 cze 2026, 00:10	4497,00 PLN	OPLACONE	Szczegóły
78d16724-8359-4964-bf9e-1cd9520288cf	5 cze 2026, 00:10	259,98 PLN	OPLACONE	Szczegóły
d44a1b96-b45c-4112-bfa4-65b89f8b5166	5 cze 2026, 00:10	237,00 PLN	OPLACONE	Szczegóły
c8c4383f-1082-475a-9f71-010d9c8c0e32	5 cze 2026, 00:10	259,98 PLN	OPLACONE	Szczegóły
013a7d87-594b-42c1-99d8-97fbf0deaf7	5 cze 2026, 00:10	237,00 PLN	OPLACONE	Szczegóły
9dc8aac1-4a2f-4058-9cf6-9e53478c3d3f	5 cze 2026, 00:10	4497,00 PLN	ANULOWANE	Szczegóły

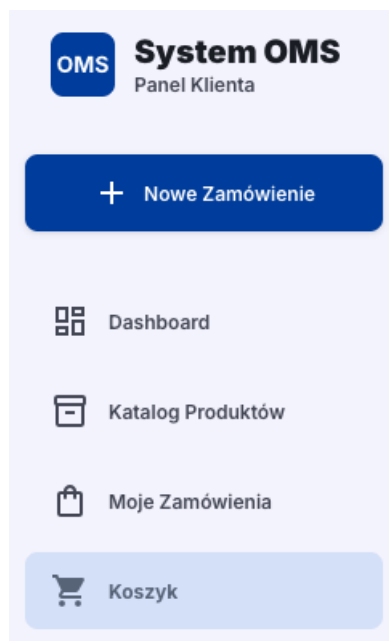
Ilustracja 7.6: Widok historii zamówień zrealizowanego serwisu.

Pulpit (DashboardView) pokazuje podsumowanie. Czyli liczbę aktywnych zamówień i ostatnią płatność.



Ilustracja 7.7: Widok pulpitu zrealizowanego serwisu.

Całość spina powłoka aplikacji ze stałym panelem nawigacyjnym.



Ilustracja 7.8: Widok panelu nawigacyjnego zrealizowanego serwisu.

7.3 Warstwa komunikacji z API

Implementacje HTTP oparto na natywnym interfejsie `fetch`, bez zewnętrznych bibliotek. Odpowiedzi backendu mapowane są na jawnie zadeklarowane typy aplikacji. Błędy mapowane są na komunikaty czytelne dla użytkownika. Nagłówki uwierzytelniające dostarcza wstrzykiwana funkcja zwrotna powiązana z dostawcą tożsamości, dzięki czemu warstwa HTTP nie zna

szczegółów mechanizmu logowania. Składanie zamówienia generuje po stronie przeglądarki klucz idempotencji (listing 7.1), współpracując z mechanizmem opisanym w sekcji 6.2.

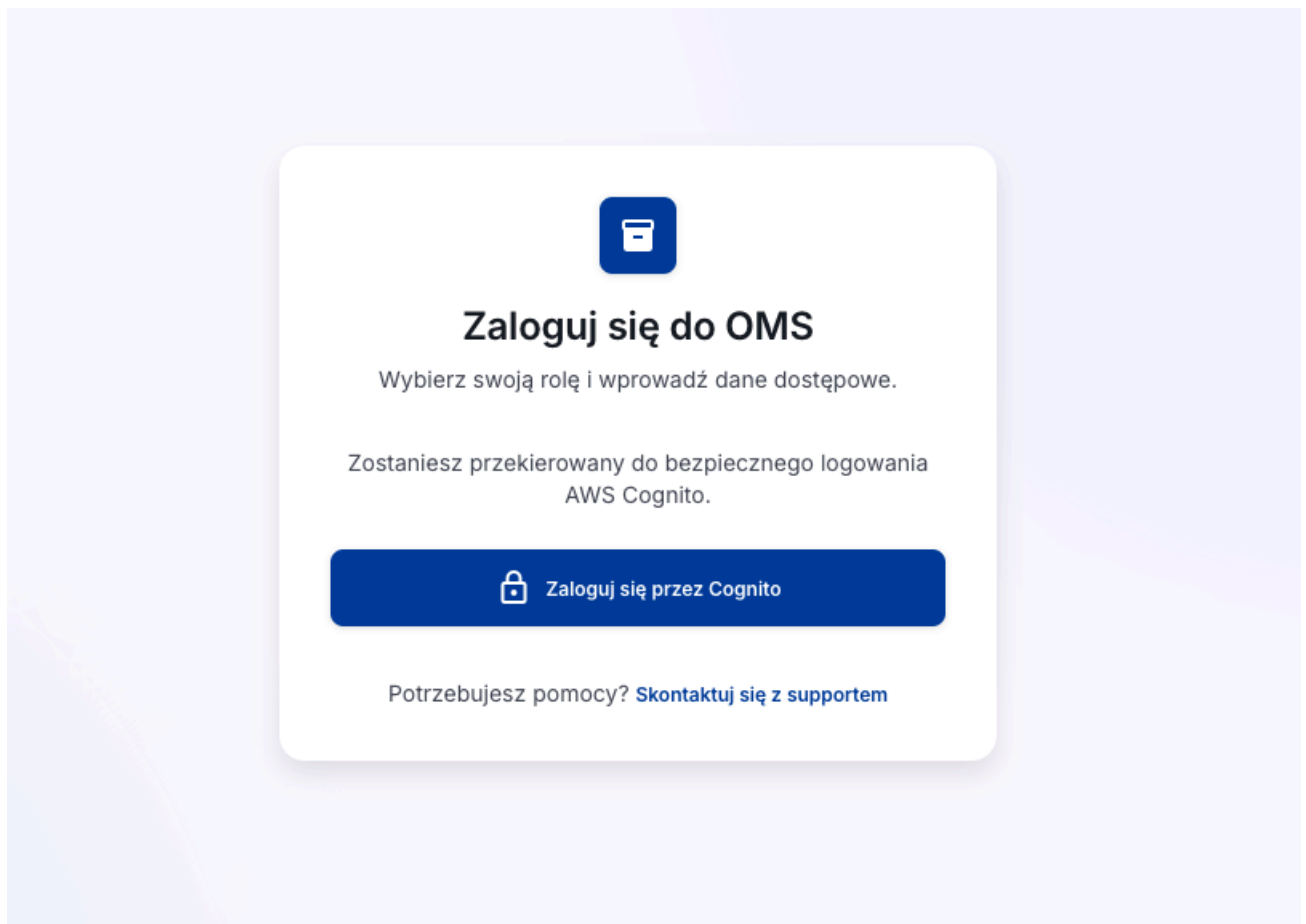
```
1 async create(req: CreateOrderRequest): Promise<OrderDetail> {
2   const res = await fetch(this.basePath, {
3     method: 'POST',
4     headers: {
5       Accept: 'application/json',
6       'Content-Type': 'application/json',
7       'Idempotency-Key': crypto.randomUUID(),
8       ...this.authHeaders(),
9     },
10    body: JSON.stringify(body),
11  })
12  if (!res.ok) throw new Error(`Nie udało się złożyć zamówienia (HTTP
13    ${res.status})`)
14  return toOrderDetail((await res.json()) as BackendOrder)
15 }
```

Listing 7.1: Złożenie zamówienia z kluczem idempotencji (fragment http.ts)

Adresy API są względne (/api/orders, /api/products, /api/v1/payments). Portal nie zna topologii backendu, a rozdziałem żądań pomiędzy mikroserwisami zajmuje się warstwa brzegowa systemu (rozdział 8). Eliminuje to problemy współdzielenia zasobów między domenami (CORS).

7.4 Logowanie użytkownika — OAuth 2.0 z PKCE

Logowanie zaimplementowano zgodnie z projektem przedstawionym w sekcji 5.8. W przepływie *authorization code* z rozszerzeniem PKCE, wobec hostowanego interfejsu logowania Cognito. Rozpoczynając logowanie, aplikacja generuje kryptograficznie losowy weryfikator oraz jego skrót SHA-256 (listing 7.2), korzystając wyłącznie z natywnych interfejsów Web Crypto. Bez zewnętrznych bibliotek kryptograficznych. Weryfikator, wraz z losowym parametrem *state* (chroniącym przed atakami CSRF) i docelową trasą po zalogowaniu, zapisywany jest w *sessionStorage*. Po czym przeglądarka przekierowywana jest na punkt końcowy autoryzacji Cognito z parametrami *code_challenge* i *code_challenge_method=S256*.



Ilustracja 7.9: Widok strony przekierowującej do logowania.

Ilustracja 7.10: Widok logowania Cognito.

```
1 function randomString(byteLength: number): string {
```

```

2   const bytes = new Uint8Array(byteLength)
3   crypto.getRandomValues(bytes)
4   return base64UrlEncode(bytes)
5 }
6
7 async function pkceChallenge(verifier: string): Promise<string> {
8   const digest = await crypto.subtle.digest(
9     'SHA-256',
10    new TextEncoder().encode(verifier),
11  )
12   return base64UrlEncode(new Uint8Array(digest))
13 }

```

Listing 7.2: Generowanie weryfikatora i wyzwania PKCE (Web Crypto)

Po uwierzytelnieniu Cognito zwraca przeglądarce na zarejestrowaną trasę zwrotną z kodem autoryzacyjnym. Widok `AuthCallbackView` weryfikuje zgodność parametru `state` z zapamiętanym, a następnie wymienia kod na tokeny żądaniem POST do punktu `/oauth2/token`. Przedstawiając oryginalny weryfikator, co domyka mechanizm PKCE. Otrzymane tokeny (identyfikacyjny, dostępowy i odświeżający, wraz z wyliczonym czasem wygaśnięcia) zapisywane są w `localStorage`. Dzięki czemu sesja przetrwa odświeżenie strony. Dane użytkownika odczytywane są z ładunku tokenu identyfikacyjnego. Portal celowo nie weryfikuje podpisu tokenu. Właściwa weryfikacja kryptograficzna następuje w każdym mikroserwisie przy każdym żądaniu (sekcja 6.6). Bezpieczeństwo systemu nie opiera się więc na kodzie działającym w przeglądarce. Wylogowanie usuwa tokeny z pamięci lokalnej i przekierowuje na punkt końcowy wylogowania Cognito, unieważniając również sesję po stronie dostawcy tożsamości.

Pełny przebieg tego procesu przedstawiono na rysunku 5.5 w rozdziale 5.

8. Budowa środowiska chmurowego — infrastruktura jako kod

Całość środowiska uruchomieniowego systemu zdefiniowano deklaratywnie w narzędziu Terraform. Środowisko jest w pełni odtwarzalne. Pojedyncze wykonanie `terraform apply` buduje je od zera, bez jakichkolwiek czynności ręcznych w konsoli AWS. Rozdział omawia kolejne warstwy tej infrastruktury w porządku, w jakim są tworzone. System uruchomiono w regionie `eu-central-1` (Frankfurt).

8.1 Terraform i zarządzanie stanem infrastruktury

Terraform przechowuje stan infrastruktury. W pliku stanu odwzorowano zasoby zadeklarowane w kodzie na zasoby istniejące w chmurze. W przypadku pracy zespołowej i automatyzacji musi być składowany zdalnie. Wykorzystano, więc backend S3. Stan trafia do dedykowanego bucketa z włączonym wersjonowaniem (możliwość odtworzenia wcześniejszych wersji stanu), szyfrowaniem oraz całkowitą blokadą dostępu publicznego. Równoległe modyfikacje wyklucza mechanizm blokady plików.

Projekt wykorzystuje trzech dostawców. (*Providers*) `aws` dla zasobów chmurowych. `Kubernetes` dla obiektów wewnątrz klastra. `Random` do generowania sekretów. Objęcie jednym narzędziem obu warstw pozwala wyrażać zależności pomiędzy nimi wprost w kodzie. Manifesty Kubernetes odwołują się do atrybutów zasobów AWS. Kolejność tworzenia wyznacza graf zależności Terraform. Parametry środowiska wyniesiono do zmiennych z wartościami w pliku `terraform.tfvars`. Dzięki czemu odtworzenie środowiska o innych parametrach (np. większych instancjach) sprowadza się do zmiany pliku zmiennych.

8.2 Konfiguracja sieci VPC

Sieć systemu obejmuje VPC o adresacji `10.0.0.0/16`, rozpiętą na dwie strefy dostępności. W każdej strefie wydzielono po dwie podsieci `/24`. Publiczną (`10.0.0.0/24`, `10.0.1.0/24`) oraz prywatną (`10.0.100.0/24`, `10.0.101.0/24`). Podsieci publiczne

posiadają trasę do bramy internetowej. Posiadają tylko elementy brzegowe: load balancer oraz bramę NAT. W podsieciach prywatnych działają węzły klastra i baza danych, których ruch wychodzący kierowany jest przez NAT (rysunek 8.1). Ze względów kosztowych zastosowano pojedynczą bramę NAT.

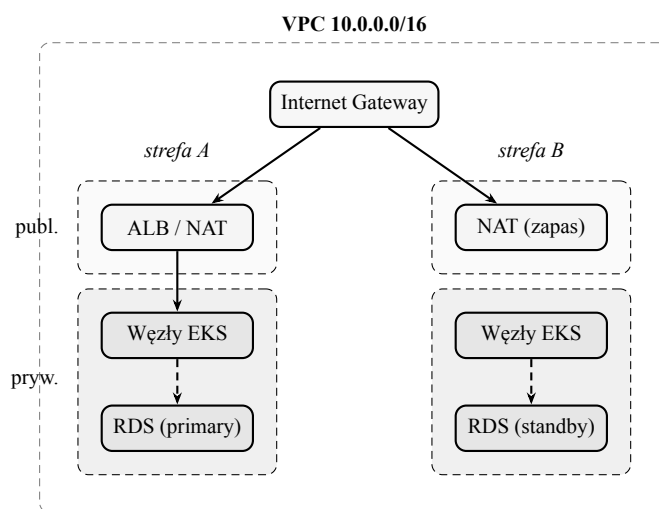
Podsieci opatrzone tagami, którymi posługuje się integracja Kubernetes z AWS przy wyborze podsieci dla load balancerów (listing 8.1). Filtrowanie ruchu realizują grupy bezpieczeństwa opisane w sekcji 8.6.

```

1 resource "aws_subnet" "private" {
2   count          = length(local.azs)
3   vpc_id         = aws_vpc.this.id
4   cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index + 100)
5   availability_zone = local.azs[count.index]
6
7   tags = {
8     Name                                = "${local.name}-
9     private-${local.azs[count.index]}"
10    "kubernetes.io/role/internal-elb"   = "1"
11    "kubernetes.io/cluster/${local.cluster_name}" = "shared"
12  }
13 }

```

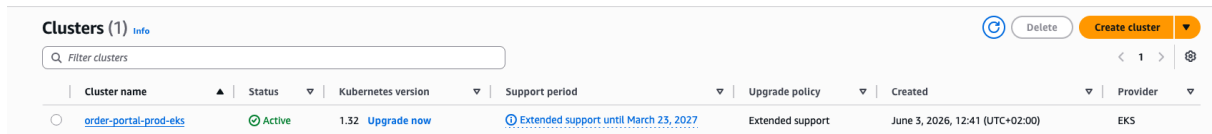
Listing 8.1: Definicja podsieci prywatnych z tagami Kubernetes (vpc.tf)



Ilustracja 8.1: Topologia sieci VPC. Komponenty brzegowe (ALB, bramy NAT) działają w podsieciach publicznych, a węzły klastra i baza danych — w prywatnych, pozbawionych bezpośredniej łączności z Internetem. Zasoby rozmieszczono w dwóch strefach dostępności. Baza danych w środowisku pracuje jednostrefowo; wariant standby zaznaczono jako opcję trybu Multi-AZ.

8.3 Utworzenie klastra EKS i grup węzłów

Klaster EKS utworzono z płaszczyzną sterowania rozpiętą na podsieci prywatne obu stref dostępności. Uprawnienia administracyjne do klastra nadawane są wpisami dostępu powiązany-
mi z tożsamościami IAM, bez ręcznej edycji map konfiguracyjnych wewnątrz klastra.



Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
order-portal-prod-eks	Active	1.32 Upgrade now	Extended support until March 23, 2027	Extended support	June 3, 2026, 12:41 (UTC+02:00)	EKS

Ilustracja 8.2: Konsola Amazon EKS: klaster `order-portal-prod-eks` w wersji Kubernetes 1.32, w stanie aktywnym.

Warstwę obliczeniową stanowi zarządzana grupa węzłów. Cztery instancje `t3.small` (2 vCPU, 2 GiB RAM każda) z obrazem AL2023, w trybie *on-demand*, z dyskami 20 GiB, rozmieszczane w podsieciach prywatnych. Aktualizacje grupy przebiegają z ograniczeniem `max_unavailable = 1`, czyli wymianą najwyżej jednego węzła naraz. Dla tak małych instancji krytyczna okazała się konfiguracja wtyczki sieciowej VPC CNI (listing 8.2). Domyślnie liczba podów na węźle ograniczona jest pulą adresów IP interfejsów sieciowych instancji co uniemożliwiłoby zmieszczenie wszystkich komponentów systemu wraz z podami systemowymi. Włączenie delegacji prefiksów (`ENABLE_PREFIX_DELEGATION`) przydziela interfejsom całe prefiksy /28 zamiast pojedynczych adresów, podnosząc limit wielokrotnie.

```
1 resource "aws_eks_addon" "vpc_cni" {
2   cluster_name      = aws_eks_cluster.this.name
3   addon_name        = "vpc-cni"
4   resolve_conflicts_on_create = "OVERWRITE"
5   resolve_conflicts_on_update = "OVERWRITE"
6
7   configuration_values = jsonencode({
8     env = {
9       ENABLE_PREFIX_DELEGATION = "true"
10      WARM_PREFIX_TARGET        = "1"
11    }
12  })
13 }
```

Listing 8.2: Dodatek VPC CNI z delegacją prefiksów (eks.tf)

Nodes (4) Info								
Filter Nodes by property or value < 1 >								
Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-10-0-100-59.eu-central-1.compute.internal	t3.small	Node group	order-portal-prod-ng	June 3, 2026, 19:03 (UTC+02:00)	Ready	29%	68%	15%
ip-10-0-100-79.eu-central-1.compute.internal	t3.small	Node group	order-portal-prod-ng	June 3, 2026, 19:03 (UTC+02:00)	Ready	34%	68%	15%
ip-10-0-101-158.eu-central-1.compute.internal	t3.small	Node group	order-portal-prod-ng	June 3, 2026, 19:03 (UTC+02:00)	Ready	21%	56%	15%
ip-10-0-101-183.eu-central-1.compute.internal	t3.small	Node group	order-portal-prod-ng	June 3, 2026, 19:03 (UTC+02:00)	Ready	33%	61%	15%

Ilustracja 8.3: Cztery węzły grupy `order-portal-prod-ng` (instancje `t3.small`) w stanie *Ready*. Wykorzystanie pamięci (56–68%) wyraźnie przewyższa wykorzystanie procesora (21–34%) — co potwierdza ustalenie z rozdziału 9, że to pamięć, a nie procesor, jest czynnikiem ograniczającym gęstość podów.

Funkcjonalność klastra dopełniają zarządzane dodatki (*add-ons*). Są to komponenty systemowe instalowane i aktualizowane przez EKS, lecz działające jako zwykłe pody wewnątrz klastra. W zrealizowanym systemie wykorzystano cztery takie dodatki. `CoreDNS` pełni rolę wewnętrznego serwera DNS klastra. To dzięki niemu mikroservisy odnajdują się nawzajem po stabilnych nazwach (np. `order-service`) zamiast po zmiennych adresach IP podów. `kube-proxy` programuje na każdym węźle reguły sieciowe realizujące obiekty `Service`, czyli kierujące ruch z nazwy usługi do jej aktualnych replik. Agent **EKS Pod Identity** umożliwia nadawanie podom uprawnień do usług AWS. Sterownik **EBS CSI** odpowiada za trwałe składowanie danych: na żądanie aplikacji (obiekt `PersistentVolumeClaim`) automatycznie tworzy w AWS wolumin EBS i podłącza go do poda.

Add-ons (6) Info						View details Edit Delete Get more add-ons	
Find add-on Any categ... Any status 6 matches < 1 2 >							
	Amazon EKS Pod Identity Agent Install EKS Pod Identity Agent to use EKS Pod Identity to grant AWS IAM permissions to pods through Kubernetes service accounts. Category: security	Status: Active	Version: v1.3.10-eksbuild.3	EKS Pod Identity: Not required	IAM role for service account (IRSA): Not required		
	Amazon VPC CNI Enable pod networking within your cluster. Category: networking	Status: Active	Version: v1.20.5-eksbuild.1	EKS Pod Identity: Not set	IAM role for service account (IRSA): Not set	Update version	
	Amazon EBS CSI Driver Enable Amazon Elastic Block Storage (EBS) within your cluster. Category: storage	Status: Active	Version: v1.60.1-eksbuild.1	EKS Pod Identity: Not set	IAM role for service account (IRSA): Not set	Update version	
	CoreDNS Enable service discovery within your cluster. Category: networking	Status: Active	Version: v1.11.4-eksbuild.33	EKS Pod Identity: Not required	IAM role for service account (IRSA): Not required	Update version	
	Metrics Server Install metrics-server to collect cluster-wide resource usage data for autoscaling and monitoring. Category: observability	Status: Active	Version: v0.8.1-eksbuild.10	EKS Pod Identity: Not required	IAM role for service account (IRSA): Not required		

Ilustracja 8.4: Zarządzane dodatki klastra EKS — agent Pod Identity, VPC CNI, sterownik EBS CSI, CoreDNS oraz metrics-server — wszystkie w stanie aktywnym.

Sterownik EBS CSI, aby tworzyć woluminy, musi mieć uprawnienia do odpowiednich operacji w API AWS. KLonto usługi sterownika powiązано mechanizmem Pod Identity z dedykowaną rolą IAM o minimalnym zakresie uprawnień (sekcja 3.5). Dzięki temu sterownik otrzymuje automatycznie rotowane poświadczenia krótkoterminowe, których ewentualny wyciek ma znikome konsekwencje, a w repozytorium ani w obrazach kontenerów nie przechowuje się żadnych trwałych kluczy.

Na potrzeby woluminów trwałych zdefiniowano klasę pamięci, wskazującą sterownikowi, jakie woluminy tworzyć. Są to woluminy typu gp3, szyfrowane w spoczynku, z możliwością późniejszego powiększenia. Zastosowano tryb wiązania *WaitForFirstConsumer*. Utworzenie woluminu jest wstrzymywane do chwili, aż Kubernetes zdecyduje, na którym węźle uruchomić korzystający z niego pod. Ma to istotne znaczenie w środowisku wielostrefowym. Wolumin EBS jest przypisany do jednej strefy dostępności i nie może być podłączony do poda działającego w innej. Odroczenie utworzenia woluminu gwarantuje, że powstanie on w tej samej strefie, co pod, który ma go używać.

8.4 Warstwa danych — RDS PostgreSQL i Secrets Manager

Bazę danych utworzono jako instancję RDS PostgreSQL 18 klasy db.t4g.micro z 20 GiB szyfrowanego składowania gp3. Wytworzono go w podsięciach prywatnych. Dostęp sieciowy ogranicza dedykowana grupa bezpieczeństwa, przepuszczająca port 5432 wyłącznie z grupy bezpieczeństwa węzłów klastra. Baza nie jest osiągalna ani z Internetu, ani z podsięci publicznych. W środowisku przyjęto jednodniową retencję kopii zapasowych i tryb jednostrefowy.

order-portal-prod-pg			
Summary			
DB identifier order-portal-prod-pg	Status 🟢 Available	Role Instance	Engine PostgreSQL
CPU 📊 3.08%	Class db.t4g.micro	Current activity 📊 0 Connections	Region & AZ eu-central-1b

Ilustracja 8.5: Konsola Amazon RDS: instancja PostgreSQL order-portal-prod-pg klasy db.t4g.micro w strefie eu-central-1b, w stanie dostępnym.

Hasło użytkownika bazy generowane jest zasobem `random_password` i nie występuje w kodzie źródłowym. Komplet poświadczeń zapisywany jest w usłudze Secrets Manager.

8.5 Wdrożenie mikroserwisów i brokera Kafka do klastra

Komponenty aplikacyjne działają w dedykowanej przestrzeni nazw `order-portal`. Deploymenty trzech mikroserwisów generowane są z jednej definicji. Pętlą po mapie usług. Zasoby serwisów ograniczono do 100m CPU / 128 MiB żądanie i 500m CPU / 512 MiB limit.

Konfiguracja każdej usługi składana jest w Terraformie do pliku `.env`, przechowywanego jako Secret i montowanego do kontenera jako wolumin tylko do odczytu. Szablony wdrożeniowe niosą również adnotacje `prometheus.io/scrape`, `prometheus.io/port` i `prometheus.io/path`, włączające pody do monitoringu (sekcja 8.7). Każdej usłudze towarzyszy obiekt Service typu ClusterIP, udostępniający ją pod stabilną nazwą DNS.

	order-service	order-portal	deployments	June 3, 2026, 13:45 (UTC+02:00)	1	1 Ready 0 Failed 1 Desired
	payment-service	order-portal	deployments	June 3, 2026, 13:45 (UTC+02:00)	1	1 Ready 0 Failed 1 Desired
	product-service	order-portal	deployments	June 3, 2026, 13:45 (UTC+02:00)	1	1 Ready 0 Failed 1 Desired

Ilustracja 8.6: Trzy mikroserwisy wdrożone w przestrzeni nazw `order-portal` jako obiekty Deployment — każdy w stanie gotowości (1 Ready / 1 Desired).

Broker Kafka uruchomiono jako Deployment z obrazem `apache/kafka:4.3.0`, Dziennik zdarzeń składowany jest na trwałym woluminie 5 GiB klasy `ebs-gp3`. Z tego powodu dane przetrwają restart i przeniesienie poda. Tematy `order.events` i `product.events` tworzy jednorazowy Job inicjalizacyjny, wykonujący narzędzie `kafka-topics` po starcie brokera.

kafka

Structured view

Raw view

Details

Status

1 Desired | 1 Available | 1 Ready | 0 Pending

Created

June 3, 2026, 13:36 (UTC+02:00)

Namespace

order-portal

Selector

app=kafka

Last transition time

June 3, 2026, 21:27 (UTC+02:00)

Images

apache/kafka:4.3.0

apache/kafka:4.3.0

Strategy type

Recreate

Pods (1) [Info](#)

Name	Status	Created	IP
kafka-5f855f9fc7-4pntg	Running	June 3, 2026, 21:27 (UTC+02:00)	10.0.100.180

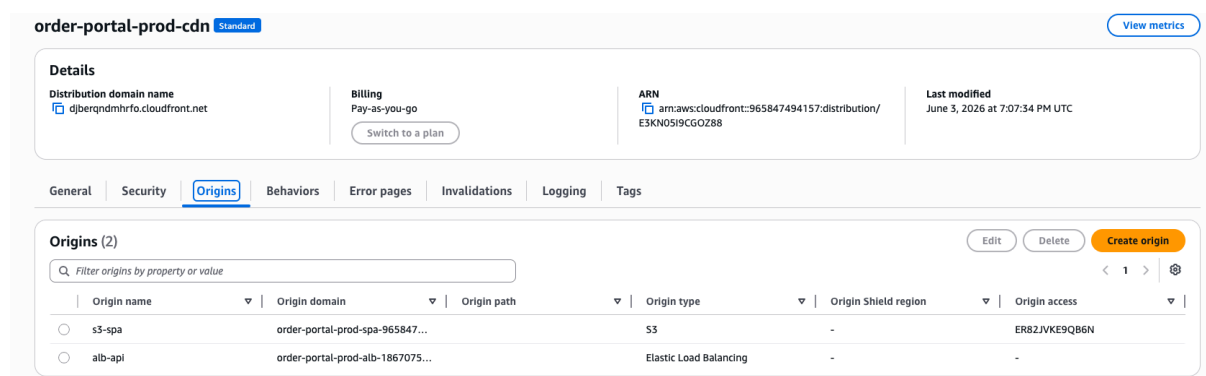
Ilustracja 8.7: Broker Apache Kafka (`apache/kafka:4.3.0`) uruchomiony w klastrze jako pojedynczy pod, ze strategią *Recreate* — zgodnie z projektem jednowęzłowego brokera w trybie KRaft.

8.6 Routing ruchu — CloudFront, ALB i reverse proxy

Jedynym publicznym punktem wejścia do systemu jest dystrybucja CloudFront z dwoma originami. Domyślne zachowanie kieruje żądania do prywatnego bucketa S3 z aplikacją webową, z polityką agresywnego buforowania i kompresją. Ponieważ trasy SPA (np. `/orders/123`) nie odpowiadają obiektom w S3, do zachowania dołączono funkcję CloudFront wykonywaną przy każdym żądaniu (listing 8.3). Żądania bez rozszerzenia pliku przepisywane są na `/index.html`, dzięki czemu głębokie odnośniki i odświeżenie strony działają poprawnie.

```
1 function handler(event) {
2   var request = event.request;
3   var uri = request.uri;
4   // No file extension (i.e. an app route, not an asset) -> serve the
      SPA.
5   if (uri === "/" || uri.indexOf(".") === -1) {
6     request.uri = "/index.html";
7   }
8   return request;
9 }
```

Listing 8.3: Funkcja CloudFront przepisująca trasy SPA (cloudfront.tf)



Ilustracja 8.8: Dystrybucja CloudFront `order-portal-prod-cdn` z dwoma originami: bucketem S3 (aplikacja SPA, dostęp przez Origin Access Control) oraz load balancerem ALB (ruch API).

Żądania pasujące do wzorców `/api/*` oraz `/grafana*` obsługuje drugi origin Application Load Balancer. Dostępu do samego ALB strzegą dwa niezależne mechanizmy. Po pierwsze, grupa bezpieczeństwa balancera dopuszcza ruch wyłącznie z zarządzanej listy prefiksów adresowych infrastruktury brzegowej CloudFront. Żądania spoza sieci CloudFront nie nawiążą nawet połączenia TCP. Po drugie, CloudFront dokleja do żądań kierowanych do ALB tajny nagłówek `X-Origin-Verify` (o wartości generowanej zasobem `random_password`), a jedyna przepuszczająca reguła nasłuchu wymaga jego obecności (listing 8.4). Żądania bez poprawne-

go nagłówka np. z cudzej dystrybucji CloudFront otrzymują odpowiedź odmowną. Wymusza to przepływ całego ruchu przez warstwę brzegową, wraz z jej terminacją TLS i ochroną przed atakami wolumetrycznymi.

Behaviors (3) Save Move up Move down Edit Delete Create behavior

Filter behaviors by property or value

	Preced...	Path pattern	Origin or origin group	Viewer protocol policy	Cache policy name	Origin request policy name	Response headers policy na...
☐	0	/api/*	alb-api	Redirect HTTP to HTTPS	Managed-CachingDisabled	Managed-AllViewerExceptHostHea	-
☐	1	/grafana*	alb-api	Redirect HTTP to HTTPS	Managed-CachingDisabled	Managed-AllViewerExceptHostHea	-
☐	2	Default (*)	s3-spa	Redirect HTTP to HTTPS	Managed-CachingOptimized	-	-

Ilustracja 8.9: Reguły zachowań (*behaviors*) CloudFront: ścieżki `/api/*` oraz `/grafana*` kierowane są do ALB z wyłączonym buforowaniem, a pozostałe — domyślnie do bucketa S3 z buforowaniem zoptymalizowanym.

```

1 resource "aws_lb_listener_rule" "from_cloudfront" {
2   listener_arn = aws_lb_listener.http.arn
3   priority     = 10
4
5   condition {
6     http_header {
7       http_header_name = "X-Origin-Verify"
8       values            = [random_password.origin_secret.result]
9     }
10  }
11
12  action {
13    type          = "forward"
14    target_group_arn = aws_lb_target_group.app.arn
15  }
16 }

```

Listing 8.4: Reguła ALB wymagająca nagłówka weryfikacyjnego (alb.tf)

Celem ALB jest grupa docelowa obejmująca węzły klastra na porcie `NodePort 30080`, pod którym działa wewnętrzne odwrócone proxy. Proxy rozdziela ruch na podstawie prefiksu ścieżki do usług klastra (listing 8.5). Odpowiada na ścieżce `/healthz` na potrzeby kontroli zdrowia ALB. Udostępnia również pod ścieżką `/grafana` interfejs Grafany. Żądania niepasujące do żadnej reguły odrzuca. Pełna droga żądania API przebiega więc: przeglądarka → CloudFront → ALB → NodePort → Proxy → Service → pod mikroserwisu, co ilustruje rysunek 8.11.

```

1 location = /healthz {
2   add_header Content-Type text/plain;
3   return 200 'ok';
4 }

```

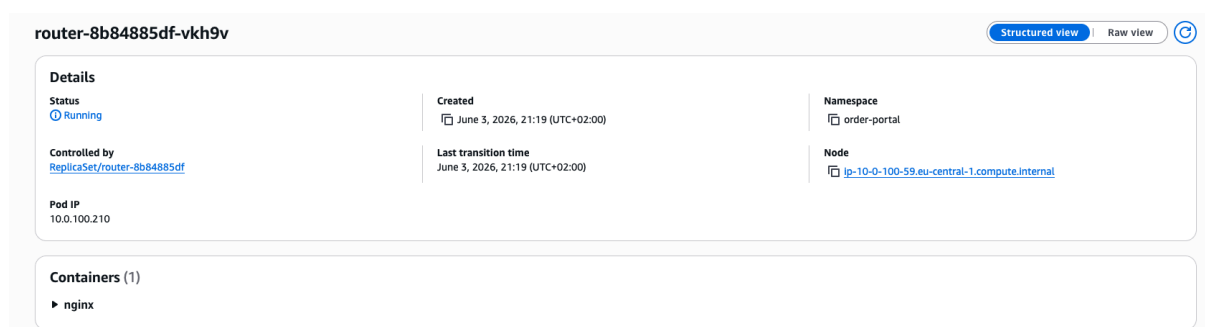


```

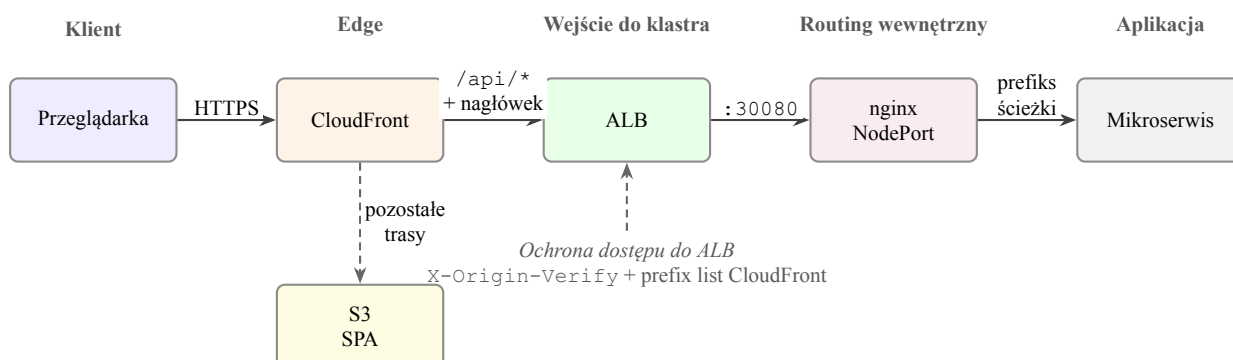
5
6 location /api/v1/payments { proxy_pass http://payment-service:9003; }
7 location /api/orders      { proxy_pass http://order-service:9002; }
8 location /api/products    { proxy_pass http://product-service:9001; }
9
10 location / {
11     add_header Content-Type text/plain;
12     return 404 'not found';
13 }

```

Listing 8.5: Reguły routingu wewnętrznego proxy nginx (fragment)



Ilustracja 8.10: Pod wewnętrznego proxy nginx (Deployment router) w przestrzeni order-portal — pojedyncza replika trasująca ruch API do mikroservisów na podstawie prefiksu ścieżki.



Ilustracja 8.11: Przepływ żądania API przez warstwy systemu. CloudFront kieruje ruch /api/* do load balancera, natomiast pozostałe trasy obsługuje bucket S3 z aplikacją SPA. Dostęp do ALB ograniczają tajny nagłówek X-Origin-Verify oraz prefix list CloudFront. Wewnętrzne proxy nginx trasuje żądanie do właściwego mikroservis na podstawie prefiksu ścieżki.

8.7 Wdrożenie monitoringu — Prometheus i Grafana

Podsystem monitoringu działa w osobnej przestrzeni nazw monitoring. Serwer Prometheus skonfigurowano z 15-sekundowym interwałem pomiarowym i dynamicznym odkrywa-

niem celów poprzez API Kubernetes. Reguły przeetykietowania (listing 8.6) pozostawiają wyłącznie pody, które jawnie zgłosiły się adnotacją `prometheus.io/scrape`. Czyli kierują pomiar na port i ścieżkę wskazane adnotacjami oraz wzbogacają metryki o etykiety przestrzeni nazw i poda. Objęcie monitoringiem nowej usługi nie wymaga więc żadnych zmian po stronie Prometheusa. Wystarczą adnotacje w jej szablonie poda. Uprawnienia do odczytu metadanych podów nadano dedykowanemu kontu usługi poprzez rolę klastrową RBAC, a baza szeregów czasowych składowana jest na trwałym woluminie `ebs-gp3` o pojemności 5 GiB.

```
1 scrape_configs:
2   - job_name: kubernetes-pods
3     kubernetes_sd_configs:
4       - role: pod
5     relabel_configs:
6       - source_labels: [
7         __meta_kubernetes_pod_annotation_prometheus_io_scrape
8         action: keep
9         regex: "true"
10      - source_labels: [
11        __meta_kubernetes_pod_annotation_prometheus_io_path
12        action: replace
13        target_label: __metrics_path__
14        regex: (.+)
15      - source_labels: [__address__,
16        __meta_kubernetes_pod_annotation_prometheus_io_port]
17        action: replace
18        regex: ([^:]+) (?:\:\\d+)?; (\\d+)
19        replacement: $1:$2
20        target_label: __address__
```

Listing 8.6: Odkrywanie celów pomiarowych na podstawie adnotacji (`prometheus.yml`)

Grafanę provisionowano w pełni deklaratywnie: źródło danych (wewnętrzny adres Prometheusa). Interfejs Grafany udostępniono pod ścieżką `/grafana` publicznej dystrybucji CloudFront (poprzez regułę proxy z sekcji 8.6), dzięki czemu wgląd w stan systemu nie wymaga dostępu administracyjnego do klastra.

Workloads: Deployments (2) View details

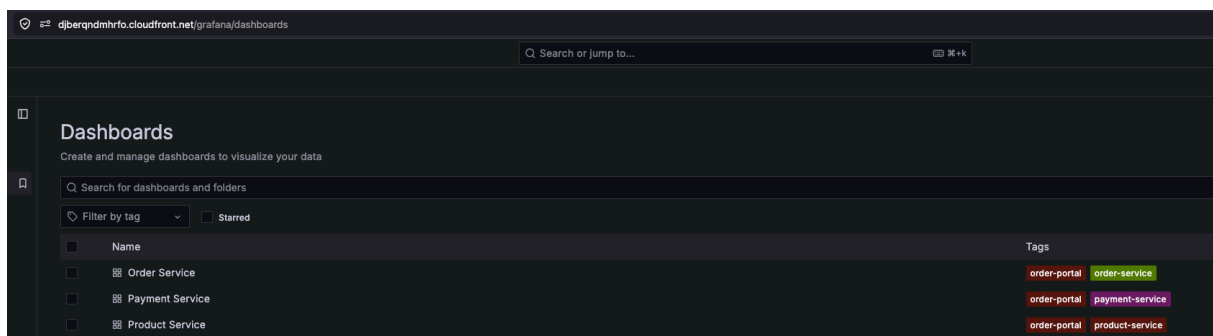
Deployment is an API object that manages a replicated application, typically by running Pods with no local state. [Learn more](#)

monitoring Filter Deployments by name

► Advanced query filters < 1 >

	Name	Namespace	Type	Created	Pod count	Status
☐	grafana	monitoring	deployments	June 3, 2026, 19:03 (UTC+02:00)	1	1 Ready 0 Failed 1 Desired
☐	prometheus	monitoring	deployments	June 3, 2026, 19:03 (UTC+02:00)	1	1 Ready 0 Failed 1 Desired

Ilustracja 8.12: Podsystem monitoringu w przestrzeni nazw `monitoring`: wdrożenia Prometheus i Grafana, oba w stanie gotowości.



Ilustracja 8.13: Interfejs Grafany udostępniony pod ścieżką `/grafana` dystrybucji CloudFront, z osobnym kokpitem dla każdego z trzech mikroservisów.

8.8 Skalowanie aplikacji i zarządzanie cyklem życia

Skalowanie poziome zdefiniowano obiektami `HorizontalPodAutoscaler` dla serwisów zamówień i produktów. Usług przyjmujących główny ruch systemu. Liczba replik zmienia się w przedziale od 1 do 2 tak, aby średnie wykorzystanie procesora utrzymywało się poniżej progu zadanego jako odsetek wartości żądanej (*request*). Próg ten jest parametrem infrastruktury (zmienna `hpa_cpu_target`). Wartość początkową 80% obniżono do 60% na podstawie testów obciążeniowych. Przy wyższym progu skalowanie nie wyzwalało się przed nasyceniem węzłów. Niższy próg pozwala dołożyć replikę z wyprzedzeniem, zanim rosnący ruch wyczerpie zapas mocy (proces strojenia opisano w rozdziale 9). Źródłem metryk dla HPA jest komponent `metrics-server`, który nie wchodzi w skład domyślnej instalacji EKS. Zainstalowano go jako zarządzany dodatek EKS (*community add-on*) zasobem `aws_eks_addon`, analogicznie do pozostałych dodatków klastra. Wąski przedział replik jest konsekwencją pojemności środowiska. Czworowęzłowy klaster instancji `t3.small` mieści ograniczoną liczbę podów. Zachowanie HPA pod obciążeniem zbadano w rozdziale 9.

Cykl życia aplikacji zarządzany jest przez zmiany w kodzie infrastruktury. Nowa wersja obrazu wskazywana jest zmienną `image_tag`, a `terraform apply` aktualizuje Deploy-

menty.

9. Testy i analiza działania systemu

Rozdział przedstawia weryfikację systemu wdrożonego w docelowym środowisku chmurowym. Omówiono metodykę testowania, wyniki testów funkcjonalnych oraz testy wydajnościowe i skalowalności. Rozdział zamykają analiza kosztów utrzymania środowiska oraz identyfikacja wąskich gardeł wraz z rekomendacjami optymalizacji.

9.1 Metodyka testowania

Testy wydajnościowe zrealizowano narzędziem **k6**, generującym ruch HTTP według programowalnych scenariuszy. Obciążenie kierowano na *publiczny* punkt wejścia systemu - dystrybucję CloudFront. Dzięki czemu pomiar obejmuje całą produkcyjną ścieżkę żądania (sekcja 8.6). Zdefiniowano dwa współbieżne scenariusze odzwierciedlające realny ruch. *Przeglądanie* czyli listowanie i filtrowanie katalogu, przegląd zamówień co stanowi większość żądań. *Zakup* czyli złożenie zamówienia, oczekiwanie na asynchroniczną walidację, utworzenie i finalizacja płatności. Liczbę wirtualnych użytkowników (VU) zwiększano etapowo według czterech profili zestawionych w tabeli 9.1.

Tabela 9.1: Profile testów wydajnościowych

Profil	Maks. VU	Czas	Cel
load	25	8 min	zachowanie przy ruchu nominalnym
stress	115	10 min	poszukiwanie granicy wydajności
max	280	10 min	wymuszenie automatycznego skalowania

Kluczową metryką własną jest *opóźnienie walidacji zamówienia*. Jest to czas od utworzenia zamówienia do osiągnięcia przez nią statusu `VALIDATED`, mierzony przez cykliczne odpytywanie (co 500 ms). Obejmuje on całą ścieżkę asynchronicznej choreografii. Publikację zdarzenia do Kafki, jego konsumpcję przez product-service, rezerwację stanów magazynowych i zapis statusu. Należy zaznaczyć, że rozdzielczość tego pomiaru ograniczona jest interwałem odpytywania, co omówiono przy interpretacji wyników. Stan systemu w trakcie testów obserwowano w Gra-

fanie (metryki techniczne i domenowe z Prometheusa) oraz poleceniami `kubectl` (stan HorizontalPodAutoscaler oraz zużycie zasobów podów). Profile procesora pobierano przez `pprof` (sekcja 9.4.1). Katalog zasiano produktami o wysokim stanie magazynowym, aby rezerwacje nie wyczerpały zapasów w trakcie testu.

9.2 Testy funkcjonalne systemu

Poprawność funkcjonalną zweryfikowano scenariuszami pokrywającymi główne przypadki użycia, wykonanymi zarówno manualnie przez portal, jak i masowo jako część scenariusza zakupowego testów wydajnościowych. Wyniki zestawia tabela 9.2.

Tabela 9.2: Scenariusze testów funkcjonalnych

Scenariusz	Oczekiwany wynik	Rezultat
Logowanie przez Cognito (OAuth 2.0 + PKCE)	przekierowanie, wydanie tokenu	✓
Przeglądanie i filtrowanie katalogu	lista zgodna z filtrami	✓
Złożenie zamówienia	status NEW, zdarzenie <code>order.created</code>	✓
Automatyczna walidacja i rezerwacja	przejście do VALIDATED	✓
Powtórzone żądanie z tym samym kluczem	brak duplikatu zamówienia	✓
Zamówienie produktu bez stanu	odrzućcie rezerwacji, CANCELLED	✓
Pełna płatność za zamówienie	PAID / SUCCEEDED	✓
Dostęp do cudzego zamówienia	odmowa (autoryzacja po tokenie)	✓
Próba opłacenia anulowanego zamówienia	odrzućcie (maszyna stanów)	✓

Szczególnie istotny jest masowy dowód poprawności płynący z testów wydajnościowych. W najcięższym profilu *max* system przyjął **7564 zamówienia**, z których **wszystkie** przeszły pełny cykl od utworzenia, przez walidację, po finalizację płatności. Bez ani jednego błędu HTTP, niespójności stanu czy nieudanej rezerwacji. Potwierdza to poprawność maszyny stanów, mechanizmu idempotencji oraz transakcyjnej finalizacji płatności w warunkach wysokiej współbieżności.

9.3 Testy wydajnościowe i skalowalność

Zbiorcze wyniki czterech profili przedstawia tabela 9.3. We wszystkich przebiegach odsetek błędnych odpowiedzi HTTP wyniósł **0%**, a wszystkie kryteria akceptacji (progi czasów odpowiedzi i odsetka błędów) zostały spełnione.

Tabela 9.3: Wyniki testów wydajnościowych (czasy w ms)

Profil	Żądań	Przel. [req/s]	HTTP p95	HTTP p99	Walid. med.	Zamówień
load	5 462	11,2	600	1 871	543	385
stress	19 921	32,9	118	757	542	981
max	267 643	445,5	92	185	547	7 564

9.3.1 Czasy odpowiedzi i efekt rozgrzania

Przy najwyższym obciążeniu (445 req/s, 280 VU) mediana czasu odpowiedzi API wyniosła 49 ms, a 95. percentyl 92 ms. Są to wyniki bardzo dobre, zważywszy na minimalne zasoby środowiska (cztery instancje `t3.small`).

Uwagę zwraca kontrintuicyjna zależność. Czasy odpowiedzi *maląy* wraz ze wzrostem obciążenia. Zjawisko to wynika z efektu rozgrzania. Profil *load* był pierwszym przebiegiem po wdrożeniu, wykonywanym na „zimnym” systemie z pustymi pulami połączeń do bazy danych, nieuzupełnionym cache kluczy JWKS oraz niewykorzystanym zapasem kredytów procesora instancji. Kolejne przebiegi korzystały już z rozgrzanych zasobów. Wniosek metodyczny konieczność rozgrzania systemu przed pomiarem wydajności odnotowano jako istotny dla wiarygodności tego typu testów.

9.3.2 Opóźnienie asynchronicznej walidacji

Mediana opóźnienia walidacji zamówienia okazała się *niewrażliwa* na obciążenie. Wyniosła 543, 542 i 547 ms odpowiednio w profilach *load*, *stress* i *max*, mimo czterdziestokrotnego wzrostu przepustowości. Oznacza to, że asynchroniczna ścieżka oparta na Kafce nie stała się wąskim gardłem nawet przy 445 req/s — przetwarzanie zdarzeń nadążało za ich napływem, bez narastania zaległości w kolejce.

Wartość ta wymaga interpretacji w świetle metodyki. Status zamówienia odpytywano co 500 ms, mediana bliska 540 ms oznacza, że zamówienie było już zwalidowane przy *pierwszym* odpytaniu (po ok. 500 ms snu), a rzeczywiste opóźnienie choreografii zdarzeń jest istotnie niższe niż zmierzone. Pomiar jest skwantowany rozdzielczością odpytywania. Wynik należy zatem traktować jako *górne ograniczenie* faktycznego opóźnienia, które sądząc po niewielkim

rozzrucie wartości (95. percentyl 590 ms w profilu *max*) mieści się w granicach pojedynczych dziesiątek do setki milisekund.

9.3.3 Automatyczne skalowanie pod obciążeniem

Profil *stress* (do 115 VU) system obsługiwał bez uruchomienia skalowania. Przy progu HorizontalPodAutoscaler ustawionym pierwotnie na 80% wykorzystania procesora względem żądania, obciążenie poszczególnych podów nie przekroczyło tej wartości, a zużycie procesora węzłów sięgnęło ok. 60%. Obserwacja ta skłoniła do obniżenia progu HPA do 60% (sekcja 8.8), aby skalowanie wyprzedzało nasycenie.

W profilu *max* mechanizm zadziałał zgodnie z projektem. Wraz ze wzrostem ruchu wykorzystanie procesora serwisu produktów przekroczyło próg, co wyzwoliło zwiększenie liczby replik z jednej do dwóch. Analogicznie przeskalowany został serwis zamówień. Przebieg skalowania ilustruje listing 9.1 — fragment ciągłej obserwacji stanu HPA, na którym widać przekroczenie progu i następujące po nim zwiększenie liczby replik.

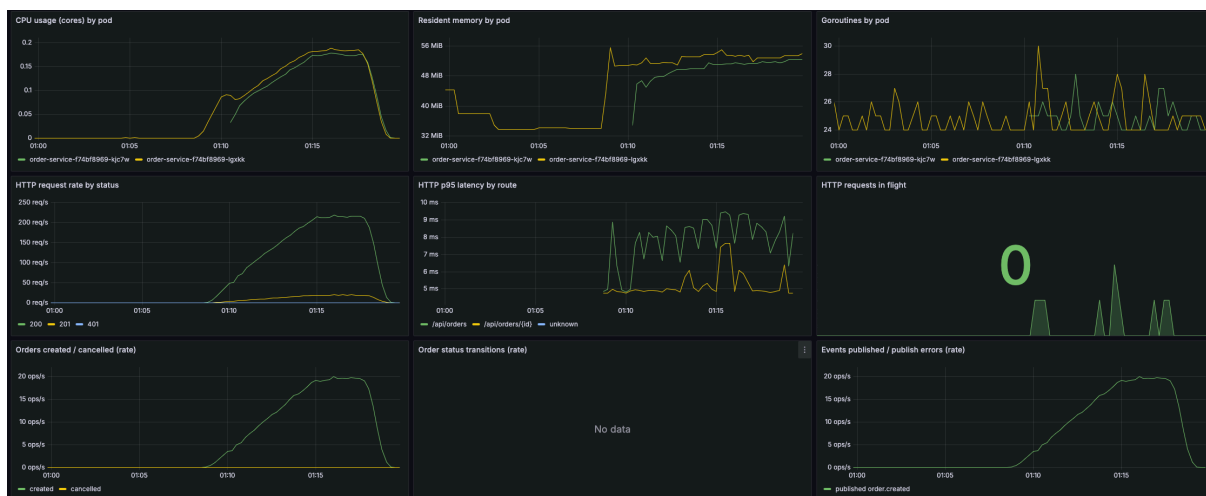
	NAME	REFERENCE	TARGETS	REPLICAS	
1	product-service	.../product-service	cpu: 41%/60%	1	
2	product-service	.../product-service	cpu: 79%/60%	1	
3	product-service	.../product-service	cpu: 109%/60%	2	<-
4	skalowanie				
5	order-service	.../order-service	cpu: 65%/60%	1	
6	order-service	.../order-service	cpu: 95%/60%	2	<-
7	skalowanie				
8	product-service	.../product-service	cpu: 264%/60%	2	<-
9	limit replik				

Listing 9.1: Reakcja HorizontalPodAutoscaler na obciążenie (profil max)

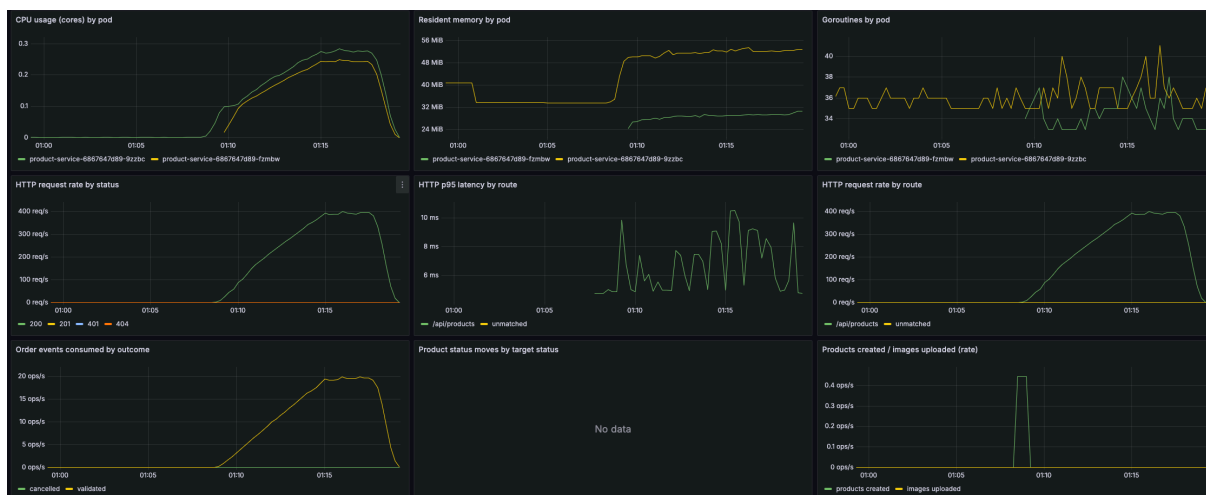
Po osiągnięciu maksymalnej liczby replik (2, ograniczonej pojemnością czterowęzłowego klastra) wykorzystanie procesora pojedynczego poda produktów rosło dalej (do ponad 260% wartości żądanej). Co unaocznia granicę skalowania poziomego przy stałej liczbie węzłów. Po wyczerpaniu limitu replik dalszy wzrost ruchu absorbowany jest już tylko przez rezerwę wydajności istniejących podów (limit CPU ustawiono pięciokrotnie powyżej żądania) Mimo to system pozostał w pełni sprawny, bez błędów i przy niskich czasach odpowiedzi. To potwierdza, że nawet zsumowane obciążenie szczytowe mieściło się w zapasie mocy obliczeniowej klastra.

Zachowanie systemu w profilu *max* ilustrują kokpity Grafany. Ilustracja 9.1 przedstawia metryki serwisu zamówień: charakterystyczne rozdwojenie wykresu zużycia procesora i pamięci w momencie pojawienia się drugiej repliki, narastającą przepustowość żądań oraz tempo tworzenia zamówień. Ilustracja 9.2 prezentuje analogiczne metryki serwisu produktów wraz z tem-

pem konsumpcji zdarzeń z Kafki.



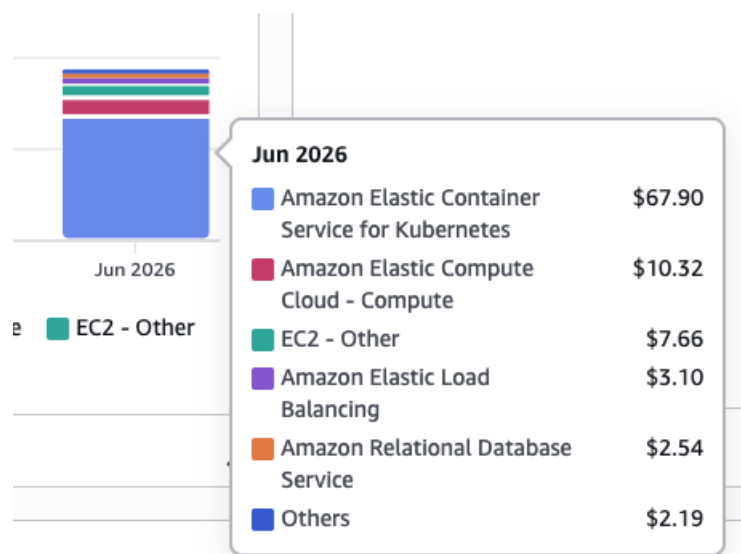
Ilustracja 9.1: Metryki serwisu zamówień w profilu *max* (Grafana): zużycie procesora i pamięci per replika, przepustowość HTTP, opóźnienia oraz tempo tworzenia zamówień. Rozdwojenie krzywych obrazuje moment automatycznego skalowania do dwóch replik.



Ilustracja 9.2: Metryki serwisu produktów w profilu *max* (Grafana): zużycie zasobów per replika, przepustowość oraz tempo konsumpcji zdarzeń z Kafki.

9.4 Analiza kosztowa wykorzystania usług AWS

Rzeczywiste koszty utrzymania środowiska, odczytane z usługi AWS Cost Explorer, przedstawiają rysunki 9.3 i 9.4.



Ilustracja 9.3: Rzeczywisty koszt środowiska w czerwcu 2026 w podziale na usługi (AWS Cost Explorer). Dominuje płaszczyzna sterowania EKS (67,90 USD) — składnik nieobjęty bezpłatnym limitem; pozostałe usługi (EC2, ELB, RDS) generują koszty rzędu pojedynczych dolarów dzięki Free Tier.

Cost summary [Info](#)

Month-to-date cost

\$93.71

↑ 353,390% compared to last month for same period

Total forecasted cost for current month

\$126.20

↑ 122,730% compared to last month's total costs

Ilustracja 9.4: Podsumowanie kosztów bieżącego miesiąca (AWS Cost Explorer): koszt narosły do dnia odczytu (93,71 USD) oraz prognoza na cały miesiąc (126,20 USD).

9.4.1 Profilowanie procesora

Profile procesora serwisów, pobrane przez `pprof` w szczycie obciążenia profilu *max*, wskazują, że obie usługi są *ograniczone operacjami wejścia–wyjścia*, nie zaś mocą obliczeniową. Dominującą pozycją w obu profilach są wywołania systemowe (ok. 18% próbek dla serwisu zamówień, ok. 22% dla produktów) związane z komunikacją sieciową oraz zapytaniem do bazy danych. Całkowite zużycie procesora w oknie pomiaru pozostawało niskie. Oznacza to, że kod aplikacyjny w języku Go nie jest czynnikiem ograniczającym wydajność.

JSON (ok. 8% próbek w serwisie produktów). Obie wskazują konkretne kierunki optymalizacji, gdyby system stał się ograniczony procesorem. Buforowanie wyniku weryfikacji tokenu na czas jego ważności oraz ewentualna zamiana refleksyjnego kodowania JSON na generowany statycznie.

9.4.2 Zidentyfikowane wąskie gardła

Analiza zachowania systemu pod obciążeniem oraz jego architektury pozwala wskazać następujące ograniczenia i rekomendacje:

- **Pamięć węzłów, nie procesor.** W spoczynku węzły wykorzystują 53–76% pamięci przy znikomym zużyciu procesora; głównym konsumentem pamięci jest broker Kafka (ok. 570 MiB), podczas gdy mikroserwisy Go zużywają zaledwie po 10–35 MiB. Na instancjach `t3.small` (2 GiB) to pamięć, a nie procesor, ogranicza gęstość podów. Co oznacza, że skalowane przez HPA według procesora repliki mogą w skrajnym przypadku nie znaleźć węzła z wolną pamięcią.
- **Wewnętrzne proxy jako pojedynczy punkt.** Pod nginx pełniący rolę reverse proxy (sekcja 8.6) działa w jednej replice i w szczycie obciążenia zużywał ok. 220 m CPU. Porównywalnie z pojedynczą repliką mikroserwisu. Stanowi on zarówno pojedynczy punkt awarii, jak i potencjalne wąskie gardło. Rekomendacją jest jego replikacja i objęcie automatycznym skalowaniem.
- **Pojedynczy broker Kafka.** Broker działa w jednej replice z woluminem o dostępie wyłącznym (sekcja 8.5). Jego awaria wstrzymuje całą asynchroniczną walidację zamówień. W środowisku produkcyjnym wymagałby klastra wielowęzłowego lub usługi zarządzanej (Amazon MSK).
- **Współdzielona instancja bazy danych.** Wszystkie serwisy korzystają z jednej instancji RDS (sekcja 5.3), co przy rosnącym obciążeniu czyni ją wspólnym punktem rywalizacji o zasoby. Wraz z transakcyjną finalizacją płatności obejmującą tabele wielu domen co jest odstępstwem od wzorca *database-per-service*.
- **Stała liczba węzłów.** Grupa węzłów pracuje w konfiguracji stałej, bez automatycznego skalowania klastra. Po wyczerpaniu limitu replik podów dalszy wzrost ruchu nie jest kompensowany dokładaniem węzłów. Wdrożenie mechanizmu skalowania klastra (np. Karpenter) zniosłoby to ograniczenie.

Żadne z powyższych ograniczeń nie ujawniło się jako problem w zakresie przeprowadzonych testów. System obsłużył 445 ządań na sekundę bezbłędnie. Stanowią one jednak naturalne

kierunki rozwoju przy podnoszeniu środowiska do poziomu produkcyjnego, podjęte w rozdziale 10.

10. Wnioski i podsumowanie

Rozdział podsumowuje pracę: ocenia stopień realizacji postawionych celów, omawia najważniejsze trudności napotkane w trakcie wytwarzania systemu wraz z przyjętymi rozwiązaniami oraz wskazuje kierunki dalszego rozwoju.

10.1 Stopień realizacji celów pracy

Cel główny pracy - zaprojektowanie, implementacja oraz wdrożenie w chmurze Amazon Web Services systemu obsługi zamówień opartego na architekturze mikroserwisowej, został osiągnięty. Powstał działający system, którego poprawność i wydajność zweryfikowano w docelowym środowisku chmurowym (rozdział 9). Realizację celów szczegółowych, postawionych w sekcji 1.1, podsumowuje tabela 10.1.

Tabela 10.1: Realizacja celów szczegółowych pracy

Cel szczegółowy	Realizacja
Analiza teoretycznych podstaw mikroservisów i technologii chmurowych	rozdz. 2–4
Projekt modelu domenowego, architektury i kontraktów komunikacyjnych	rozdz. 5
Implementacja mikroservisów (zamówienia, produkty, płatności)	rozdz. 6
Implementacja aplikacji webowej (portal)	rozdz. 7
Budowa środowiska chmurowego jako infrastruktury jako kod	rozdz. 8
Testy systemu oraz analiza działania i kosztów	rozdz. 9

Zakres pracy zrealizowano zgodnie z założeniami, z jednym świadomym ograniczeniem. Spośród pierwotnie rozważanych usług pomocniczych zaimplementowano trzy serwisy realizujące rdzeń procesu zakupowego, pozostawiając usługi powiadomień i analityki jako kierunek dalszego rozwoju (sekcja 10.3). Decyzja ta pozwoliła skupić wysiłek na kompletności pionowej systemu. Od interfejsu użytkownika, przez logikę biznesową, po infrastrukturę. Zamiast na zwielokrotnianiu podobnych do siebie usług.

10.2 Napotkane trudności i sposoby ich rozwiązania

Wytwarzanie systemu wiązało się z szeregiem problemów, których rozwiązania okazały się pouczające.

Limit adresów IP na węzłach. Dobór niewielkich instancji `t3.small` ze względów kosztowych zderzył się z ograniczeniem wtyczki sieciowej VPC CNI. Domyślnie węzeł takiej klasy mieści zaledwie 11 podów, co uniemożliwiało uruchomienie wszystkich komponentów systemu. Problem rozwiązano włączeniem delegacji prefiksów (sekcja 8.3), wielokrotnie podnoszącej ten limit.

Brak komponentu `metrics-server`. Automatyczne skalowanie oparte na zużyciu procesora wymaga komponentu `metrics-server`, który nie wchodzi w skład domyślnej instalacji EKS. Wobec czego HorizontalPodAutoscaler pozostawał bezczynny, raportując nieznanе wykorzystanie zasobów. Zachowując zasadę utrzymania całości infrastruktury jako kod, komponent zainstalowano jako zarządzany dodatek EKS, a nie ręcznie (sekcja 8.8).

Strojenie progu skalowania. Pierwotny próg HPA na poziomie 80% wykorzystania procesora okazał się zbyt wysoki. W teście *stress* skalowanie nie uruchomiło się mimo znaczącego obciążenia (sekcja 9.3.3). Na podstawie obserwacji próg obniżono do 60%, tak aby dokładanie replik wyprzedzało nasycenie. Co zilustrowało wartość empirycznego strojenia parametrów wobec przyjmowania ich „na wyczucie”.

Bezpieczne odizolowanie warstwy aplikacyjnej. Wystawienie systemu na świat wyłącznie przez CloudFront, przy jednoczesnym uniemożliwieniu dostępu do load balancera z pominięciem warstwy brzegowej, wymagało połączenia dwóch mechanizmów. Ograniczenia grupy bezpieczeństwa do zakresów adresowych CloudFront oraz tajnego nagłówka weryfikacyjnego wstrzykiwanego przez CDN i wymaganego przez regułę load balancera (sekcja 8.6).

Spójność danych w systemie rozproszonym. Utrzymanie spójności stanu rozproszonego pomiędzy usługami rozwiązano dwojako. Warunkowymi, niepodzielnymi operacjami bazodanowymi odpornymi na współbieżność (sekcje 6.3, 6.4). Oraz idempotentnym przetwarzaniem zarówno żądań HTTP, jak i zdarzeń Kafki.

10.3 Rekomendacje dotyczące dalszego rozwoju systemu

Zrealizowany system stanowi kompletną, działającą całość, lecz jako projekt o charakterze demonstracyjnym pozostawia przestrzeń do rozwoju w kierunku wdrożenia produkcyjnego. Najistotniejsze rekomendacje są następujące.

- **Domknięcie choreografii zdarzeń.** Należy uczynić order-service konsumentem tematu `product.events`, eliminując przejściowe rozwiązanie, w którym product-service zapisuje status zamówienia bezpośrednio (sekcja 6.3). Co przywróci pełną wyłączność własności danych.
- **Rozdzielenie warstwy danych.** Przydzielenie każdej usłudze własnej bazy oraz zastąpienie transakcyjnej, międzydomenowej finalizacji płatności wzorcem sagi przywróciłoby pełną zgodność ze wzorcem *database-per-service*.
- **Wysoka dostępność komponentów stanowych.** Pojedynczy broker Kafka oraz jednostrefowa baza danych powinny zostać zastąpione konfiguracjami wysokodostępными (klastery Kafka lub Amazon MSK, tryb Multi-AZ RDS), a wewnętrzne proxy zreplikowane i objęte autoskalowaniem.
- **Rozbudowa funkcjonalna.** Naturalnym rozszerzeniem jest implementacja usług powiadomień i analityki. Dzięki architekturze sterowanej zdarzeniami, mogą dołączyć jako kolejni konsumenci istniejących zdarzeń bez zmian w usługach produkujących.
- **Optymalizacje wydajnościowe i kosztowe.** W razie wzrostu obciążenia procesora wskazane jest buforowanie wyniku weryfikacji tokenów JWT (sekcja 9.4.1).

10.4 Podsumowanie końcowe

W ramach pracy zaprojektowano, zaimplementowano i wdrożono w chmurze publicznej kompletny system obsługi zamówień oparty na architekturze mikroserwisowej. Powstał system złożony z trzech mikroserwisów w języku Go, komunikujących się synchronicznie poprzez REST oraz asynchronicznie poprzez zdarzenia domenowe. Powstał z aplikacji webowej Vue, oraz brokera komunikatów. Wszystkie te komponenty uruchomionych na klastrze Kubernetes w usłudze Amazon EKS. Moduł perzystencji danych został postawiony w usłudze Amazon RDS. Infrastruktura została zdefiniowana w całości jako kod i objęta monitoringiem.

Praca wykazała, że zbudowanie tej klasy systemu rozproszonego skalowalnego, obserwowalnego i bezpiecznego jest dziś w zasięgu pojedynczego inżyniera. Głównym kosztem architektury mikroserwisowej pozostaje nie tyle implementacja samych usług, co złożoność operacyjna ich środowiska. Sieci, orkiestracji, wdrażania i obserwowalności. To właśnie zarządzane usługi chmurowe i podejście infrastruktury jako kod okazały się czynnikiem, który tę złożoność czyni możliwą do opanowania. Przeprowadzone testy potwierdziły, że zrealizowany system mimo celowo minimalnych zasobów, obsługuje setki żądań na sekundę bezbłędnie. Jego architektura, wraz ze wskazanymi kierunkami rozwoju, stanowi solidną podstawę do ewentualnego wdrożenia produkcyjnego.

Spis ilustracji

5.1	Diagram przypadków użycia systemu. Klient realizuje pełny proces zakupu, administrator zarządza katalogiem produktów; logowanie jest wspólne dla obu ról.	31
5.2	Maszyna stanów cyklu życia zamówienia. Linia ciągłą oznaczono ścieżkę główną, przerywaną — przejścia do stanów terminalnych CANCELLED i FAILED, osiągalne z każdego stanu nieterminalnego (dla czytelności pokazano reprezentatywne krawędzie). Stany terminalne wyróżniono szarym tłem.	32
5.3	Architektura systemu. Linia ciągła — komunikacja synchroniczna (HTTPS/REST), linia przerywana — asynchroniczna wymiana zdarzeń przez Apache Kafka. Cały ruch użytkownika przechodzi przez warstwę brzegową (CloudFront + ALB + nginx), a uwierzytelnianie realizuje Amazon Cognito. Komponenty walidacji JWT po stronie serwisów pominięto dla czytelności. . .	33
5.4	Diagram sekwencji przepływu zakupowego. Po synchronicznym utworzeniu zamówienia jego walidacja przebiega asynchronicznie, według choreografii zdarzeń Kafki (order-service → product-service); płatność i jej finalizacja są ponownie operacjami synchronicznymi.	36
5.5	Przebieg uwierzytelniania OAuth 2.0 <i>authorization code</i> z rozszerzeniem PKCE. Portal generuje weryfikator i jego skrót; po zalogowaniu wymienia kod autoryzacyjny — wraz z weryfikatorem — na tokeny JWT. Każdy mikroserwis weryfikuje podpis tokenu samodzielnie, na podstawie kluczy publicznych Cognito (JWKS).	40
7.1	Widok katalogu produktów zrealizowanego serwisu.	50
7.2	Widok koszyka zrealizowanego serwisu.	50
7.3	Widok finalizacji zamówienia zrealizowanego serwisu.	51

7.4	Widok szczegółów zamówienia zrealizowanego serwisu.	51
7.5	Widok płatności zrealizowanego serwisu.	52
7.6	Widok historii zamówień zrealizowanego serwisu.	52
7.7	Widok pulpitu zrealizowanego serwisu.	53
7.8	Widok panelu nawigacyjnego zrealizowanego serwisu.	53
7.9	Widok strony przekierowującej do logowania.	55
7.10	Widok logowania Cognito.	55
8.1	Topologia sieci VPC. Komponenty brzegowe (ALB, bramy NAT) działają w podsieciach publicznych, a węzły klastra i baza danych — w prywatnych, pozbawionych bezpośredniej łączności z Internetem. Zasoby rozmieszczono w dwóch strefach dostępności. Baza danych w środowisku pracuje jedno-strefowo; wariant standby zaznaczono jako opcję trybu Multi-AZ.	58
8.2	Konsola Amazon EKS: klaster <code>order-portal-prod-eks</code> w wersji Kubernetes 1.32, w stanie aktywnym.	59
8.3	Cztery węzły grupy <code>order-portal-prod-ng</code> (instancje <code>t3.small</code>) w stanie <i>Ready</i> . Wykorzystanie pamięci (56–68%) wyraźnie przewyższa wykorzystanie procesora (21–34%) — co potwierdza ustalenie z rozdziału 9, że to pamięć, a nie procesor, jest czynnikiem ograniczającym gęstość podów.	60
8.4	Zarządzane dodatki klastra EKS — agent Pod Identity, VPC CNI, sterownik EBS CSI, CoreDNS oraz metrics-server — wszystkie w stanie aktywnym. . . .	60
8.5	Konsola Amazon RDS: instancja PostgreSQL <code>order-portal-prod-pg</code> klasy <code>db.t4g.micro</code> w strefie <code>eu-central-1b</code> , w stanie dostępnym. . .	61
8.6	Trzy mikroserwisy wdrożone w przestrzeni nazw <code>order-portal</code> jako obiekty Deployment — każdy w stanie gotowości (1 Ready / 1 Desired). . . .	62
8.7	Broker Apache Kafka (<code>apache/kafka:4.3.0</code>) uruchomiony w klastrze jako pojedynczy pod, ze strategią <i>Recreate</i> — zgodnie z projektem jednowęzłowego brokera w trybie KRaft.	62
8.8	Dystrybucja CloudFront <code>order-portal-prod-cdn</code> z dwoma originami: bucketem S3 (aplikacja SPA, dostęp przez Origin Access Control) oraz load balancerem ALB (ruch API).	63

8.9	Reguły zachowań (<i>behaviors</i>) CloudFront: ścieżki <code>/api/*</code> oraz <code>/grafana</code> * kierowane są do ALB z wyłączonym buforowaniem, a pozostałe — domyślnie do bucketa S3 z buforowaniem zoptymalizowanym.	64
8.10	Pod wewnętrznego proxy nginx (Deployment router) w przestrzeni <code>order-portal</code> — pojedyncza replika trasująca ruch API do mikroserwisów na podstawie prefiksu ścieżki.	65
8.11	Przepływ żądania API przez warstwy systemu. CloudFront kieruje ruch <code>/api/*</code> do load balancera, natomiast pozostałe trasy obsługuje bucket S3 z aplikacją SPA. Dostęp do ALB ograniczają tajny nagłówek <code>X-Origin-Verify</code> oraz prefix list CloudFront. Wewnętrzne proxy nginx trasuje żądanie do właściwego mikroserwisu na podstawie prefiksu ścieżki.	65
8.12	Podsystem monitoringu w przestrzeni nazw <code>monitoring</code> : wdrożenia Prometheus i Grafana, oba w stanie gotowości.	67
8.13	Interfejs Grafany udostępniony pod ścieżką <code>/grafana</code> dystrybucji CloudFront, z osobnym kokpitem dla każdego z trzech mikroserwisów.	67
9.1	Metryki serwisu zamówień w profilu <i>max</i> (Grafana): zużycie procesora i pamięci per replika, przepustowość HTTP, opóźnienia oraz tempo tworzenia zamówień. Rozdwojenie krzywych obrazuje moment automatycznego skalowania do dwóch replik.	73
9.2	Metryki serwisu produktów w profilu <i>max</i> (Grafana): zużycie zasobów per replika, przepustowość oraz tempo konsumpcji zdarzeń z Kafki.	73
9.3	Rzeczywisty koszt środowiska w czerwcu 2026 w podziale na usługi (AWS Cost Explorer). Dominuje płaszczyzna sterowania EKS (67,90 USD) — składnik nieobjęty bezpłatnym limitem; pozostałe usługi (EC2, ELB, RDS) generują koszty rzędu pojedynczych dolarów dzięki Free Tier.	74
9.4	Podsumowanie kosztów bieżącego miesiąca (AWS Cost Explorer): koszt narosły do dnia odczytu (93,71 USD) oraz prognoza na cały miesiąc (126,20 USD).	74
9.5	Flamegraph profilu procesora serwisu zamówień (<code>pprof</code>), zarejestrowany w szczycie obciążenia profilu <i>max</i> . Szerokość słupka odpowiada udziałowi funkcji w czasie procesora. Widoczne są: obsługa żądań HTTP, weryfikacja tokenu JWT (<code>CognitoVerifier.Verify</code>), zapytania do PostgreSQL przez <code>pgx</code> oraz (de)serializacja JSON. Znaczną część stanowią wywołania systemowe związane z siecią i bazą danych, co potwierdza ograniczenie operacjami wejścia-wyjścia, nie mocą obliczeniową.	75

9.6	Flamegraph profilu procesora serwisu produktów (<code>pprof</code>) w szczycie obciążenia profilu <i>max</i> . Podobnie jak w serwisie zamówień dominują wywołania systemowe (operacje sieciowe i bazodanowe), a spośród operacji obliczeniowych wyróżniają się weryfikacja JWT oraz kodowanie JSON — profil potwierdza, że usługa jest ograniczona wejściem–wyjściem.	75
-----	--	----

Spis tabel

2.1	Porównanie architektury monolitycznej i mikroserwisowej (opracowanie własne na podstawie [1], [2])	13
5.1	Stany cyklu życia zamówienia	32
5.2	Punkty końcowe REST mikroserwisów	34
9.1	Profile testów wydajnościowych	69
9.2	Scenariusze testów funkcjonalnych	70
9.3	Wyniki testów wydajnościowych (czasy w ms)	71
10.1	Realizacja celów szczegółowych pracy	78

Spis listingów

5.1	Struktura zdarzenia order.created	35
5.2	Schemat tabeli orders	37
5.3	Schemat tabeli idempotency keys	37
5.4	Schemat tabeli products	37
5.5	Schemat tabeli images	38
5.6	Schemat tabeli payments	38
6.1	Struktura katalogów serwisu zamówień	41
6.2	Implementacja maszyny stanów zamówienia	42
6.3	Atomowe zajęcie klucza idempotencji	43
6.4	Warunkowe przesunięcie ilości w transakcji płatności	45
6.5	Weryfikacja tokenu JWT wobec kluczy Cognito	46
6.6	Wieloetapowy Dockerfile serwisu zamówień	47
7.1	Złożenie zamówienia z kluczem idempotencji (fragment http.ts)	54
7.2	Generowanie weryfikatora i wyzwania PKCE (Web Crypto)	55
8.1	Definicja podsieci prywatnych z tagami Kubernetes (vpc.tf)	58
8.2	Dodatek VPC CNI z delegacją prefiksów (eks.tf)	59
8.3	Funkcja CloudFront przepisująca trasy SPA (cloudfront.tf)	63
8.4	Reguła ALB wymagająca nagłówka weryfikacyjnego (alb.tf)	64
8.5	Reguły routingu wewnętrznego proxy nginx (fragment)	64
8.6	Odkrywanie celów pomiarowych na podstawie adnotacji (prometheus.yml)	66
9.1	Reakcja HorizontalPodAutoscaler na obciążenie (profil max)	72

Bibliografia

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2 wyd. Sebastopol, CA: O'Reilly Media, 2021, ISBN: 978-1-492-03402-5.
- [2] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018, ISBN: 978-1-61729-454-9.
- [3] B. Burns, J. Beda, K. Hightower i L. Evenson, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 3 wyd. Sebastopol, CA: O'Reilly Media, 2022, ISBN: 978-1-098-11020-8.
- [4] „CNCF Annual Survey 2023”, Cloud Native Computing Foundation, dostęp 4 czerwca 2026. URL: <https://www.cncf.io/reports/cncf-annual-survey-2023/>
- [5] P. Mell i T. Grance, „The NIST Definition of Cloud Computing”, National Institute of Standards i Technology, Special Publication 800-145, 2011.
- [6] J. Lewis i M. Fowler. „Microservices: a definition of this new architectural term”, dostęp 4 czerwca 2026. URL: <https://martinfowler.com/articles/microservices.html>
- [7] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2003, ISBN: 978-0-321-12521-7.
- [8] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media, 2017, ISBN: 978-1-449-37332-0.
- [9] A. Wiggins. „The Twelve-Factor App”, dostęp 4 czerwca 2026. URL: <https://12factor.net/>
- [10] „Kubernetes Documentation”, The Kubernetes Authors, dostęp 4 czerwca 2026. URL: <https://kubernetes.io/docs/>
- [11] „Prometheus Documentation”, Prometheus Authors, dostęp 4 czerwca 2026. URL: <https://prometheus.io/docs/>
- [12] „AWS Documentation”, Amazon Web Services, dostęp 4 czerwca 2026. URL: <https://docs.aws.amazon.com/>

- [13] „How Go Became the Language of the Cloud”, Medium, dostęp 4 czerwca 2026. URL: <https://medium.com/@yashbatra11111/how-go-became-the-language-of-the-cloud-fa3b09bd4092>
- [14] D. Hardt, „The OAuth 2.0 Authorization Framework”, Internet Engineering Task Force, RFC 6749, 2012.
- [15] N. Sakimura, J. Bradley i N. Agarwal, „Proof Key for Code Exchange by OAuth Public Clients”, Internet Engineering Task Force, RFC 7636, 2015.